

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Федеральное государственное бюджетное образовательное учреждение
высшего образования**
«Поволжский государственный университет телекоммуникаций и информатики»

Факультет	<u>Кибербезопасности и управления</u>
Направление подготовки / специальность	<u>09.03.04 Программная инженерия</u>
Кафедра	<u>Программной инженерии</u>

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)**

**Разработка мобильного приложения для улучшения
продуктивности и качества жизни**

Утверждаю	<u>и.о. зав.каф.</u>	<u>к.т.н.</u>			<u>И.С. Макаров</u>
	Должность	Уч.степень, звание	Подпись	Дата	Инициалы Фамилия
Руководитель	<u>ст. препод.</u>				<u>Е.В. Расеева</u>
Н. контролер	<u>ст. препод.</u>				<u>С.В. Чернова</u>
Разработал	<u>РПИС-11</u>				<u>М.Л. Ларкин</u>
	Группа		Подпись	Дата	Инициалы Фамилия

Самара, 2025

Содержание

Задание.....	4
Отзыв руководителя.....	6
Показатели качества ВКР	8
Реферат	9
Введение	10
1 Анализ предметной области и теоретические основы	12
1.1 История планировщиков, трекеров и продуктивности	12
1.2 Геймификация как инструмент повышения вовлечённости	13
1.3 Психология мотивации: внутренняя и внешняя	15
1.4 Обзор и сравнение аналогов	17
1.5 Архитектура Android-приложений и UX-подходы	19
1.6 Обоснование технологий и библиотек.....	21
1.7 Выводы по главе.....	23
2 Проектирование и реализация мобильного приложения “Nabochi”	25
2.1 Постановка задачи.....	25
2.2 Архитектура приложения	28
2.3 Модель данных и база данных.....	30
2.4 Реализация трекинга привычек и задач.....	32
2.5 Система ХР и уровней.....	39
2.6 Жизни персонажа и магазин	41
2.7 Достижения (ачивки).....	44
2.8 Магазин кастомизации	46
2.9 Авторизация и хранение аккаунта.....	49
2.10 Статистика	51
2.11 Пользовательский интерфейс	54
2.12 Выводы по главе.....	61

3 Управленческий анализ проекта “Nabochi”	63
3.1 Жизненный цикл проекта и этапы разработки	63
3.2 SWOT-анализ проекта Nabochi	65
3.3 GROW-модель	66
3.4 GAP-анализ	68
3.5 Матрица МакКинзи	69
3.6 Метрики успеха проекта	70
3.7 Выводы по главе	72
Заключение	73
Список использованных источников	75
Приложение А - Исходные коды программы	76
Приложение Б - Презентационный материал	81

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Поволжский государственный университет телекоммуникаций и информатики»

ЗАДАНИЕ
по подготовке выпускной квалификационной работы

Студента Ларкина Максима Леонидовича

Тема ВКР **Разработка мобильного приложения для
улучшения продуктивности и качества жизни**

Утверждена приказом по университету от 15.04.2025 № 111-2

Срок сдачи студентом законченной ВКР 05.06.2025

Исходные данные и постановка задачи

- 1) *Анализ рынка и функций мобильных приложений для продуктивности, трекинга задач и формирования привычек*
- 2) *Сбор и систематизация пользовательских требований к возможностям и интерфейсу приложения*
- 3) *Изучение и выбор современных архитектурных решений и инструментов разработки для Android*
- 4) *Проектирование структуры и архитектуры мобильного приложения на основе выбранного стека технологий*
- 5) *Разработка основных модулей приложения по улучшению продуктивности и качества жизни*
- 6) *Оценка практической значимости и перспектив развития программного продукта*

Перечень подлежащих разработке в ВКР

вопросов или краткое содержание ВКР. Сроки исполнения 12.05.2025

- 1) Анализ предметной области, исследование современных подходов к продуктивности и формированию привычек.
- 2) Обзор и сравнение существующих мобильных приложений для трекинга задач и привычек.
- 3) Формирование требований к разрабатываемому приложению, обоснование выбора архитектуры и технологий
- 4) Проектирование и реализация мобильного приложения
- 5) Разработка модуля аналитики и визуализации пользовательского прогресса.
- 6) Проведение управленческого анализа проекта (SWOT, GROW, GAP-анализ, определение метрик успеха).
- 7) Формулировка выводов и рекомендаций по дальнейшему развитию приложения.

Перечень графического материала.

Сроки исполнения 26.05.2025

Презентационный материал

Дата выдачи задания « 16 » апреля 2025 г.

Кафедра

Программной инженерии

Утверждаю

<i>и.о. зав.каф. к.т.н</i>	<i>16.04.25</i>	<i>И.С. Макаров</i>
Должность Уч.степень, звание	Подпись Дата	Инициалы Фамилия

Руководитель

<i>ст. препод.</i>	<i>16.04.25</i>	<i>Е.В. Расеева</i>
Должность Уч.степень, звание	Подпись Дата	Инициалы Фамилия

Задание принял
к исполнению

<i>РПИС-11</i>	<i>16.04.25</i>	<i>М. Л. Ларкин</i>
Группа	Подпись Дата	Инициалы Фамилия

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Поволжский государственный университет телекоммуникаций и информатики»**

ОТЗЫВ РУКОВОДИТЕЛЯ

Тип ВКР	Бакалаврская работа
Студента(ки)	Ларкина Максима Леонидовича
Направление подготовки/ специальность	Программная инженерия
Тема ВКР	Разработка мобильного приложения для улучшения продуктивности и качества жизни.
Руководитель	Расеева Е.В.
Ученая степень, звание	ст. препод.
Место работы (должность)	ПГУТИ, ст. препод. каф. При

АКТУАЛЬНОСТЬ ТЕМЫ

В условиях роста объемов информации и увеличения темпа жизни возрастает необходимость в инструментах для эффективной самоорганизации и управления личной продуктивностью. Мобильные приложения выступают основными средствами для структурирования задач, контроля выполнения и формирования полезных привычек.

В настоящей работе рассматривается задача создания мобильного приложения, обеспечивающего повышение продуктивности за счёт интеграции инструментов трекинга задач и привычек с элементами геймификации. Данное приложение может использоваться как самостоятельное решение оптимизации деятельности пользователя.

ОЦЕНКА СОДЕРЖАНИЯ РАБОТЫ

(Структура, логика и стиль изложения представленного материала. глубина и степень проработки материала, обоснованность изложенных выводов, использование математического аппарата, использование средств вычислительной техники, макетирование, моделирование, экспериментирование)

Структура работы выстроена логично и последовательно. В первой части представлен комплексный анализ современных подходов к вопросам продуктивности, трекинга задач и формирования привычек, а также выполнено сравнение существующих решений. На основе проведённого анализа обоснован выбор архитектуры и технологий для разработки мобильного приложения.

В основной части подробно рассмотрены этапы проектирования и реализации ключевых модулей приложения, приведено описание архитектурных решений, моделей данных, а также механик геймификации и аналитики. Итоговые результаты демонстрируют функциональные возможности реализованного программного продукта, его интерфейс и особенности пользовательского опыта.

Стиль изложения соответствует требованиям работ подобного направления, выводы по каждой главе обоснованы, при выполнении исследования активно использовались современные средства вычислительной техники, моделирования и макетирования пользовательских интерфейсов.

СТЕПЕНЬ ДОСТИЖЕНИЯ ЦЕЛИ И ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ

(Полнота раскрытия исследуемой темы, практическая ценность и возможность внедрения)

Тематика работы раскрыта в полном объеме, разработанное мобильное приложение для повышения продуктивности и качества жизни обладает высокой практической ценностью и может быть рекомендовано для использования.

ЗАКЛЮЧЕНИЯ ПО ПРЕДСТАВЛЕННОЙ РАБОТЕ

(Степень самостоятельной работы студента; совокупная оценка труда студента и его квалификация)

В процессе выполнения выпускной квалификационной работы Ларкин М.Л. проявил высокий уровень самостоятельности, уверенное владение современными технологиями мобильной разработки и глубокое понимание архитектурных подходов. ВКР выполнена в полном соответствии с предъявляемыми требованиями, отличается практической значимостью и актуальностью выбранной тематики. Студент Ларкин Максим Леонидович заслуживает оценки «отлично», а также присвоения квалификации «бакалавр» по направлению 09.03.04 «Программная инженерия».

Руководитель ВКР

Подпись

Дата

Е. В. Расеева

Инициалы Фамилия

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Федеральное государственное бюджетное образовательное учреждение
высшего образования**
«Поволжский государственный университет телекоммуникаций и информатики»

ПОКАЗАТЕЛИ КАЧЕСТВА ВКР

По ВКР студента

Ларкина Максима Леонидовича

На тему

**Разработка мобильного приложения для улучшения
продуктивности и качества жизни**

1 Работа выполнена:

- по теме, предложенной студентом
- по заявке предприятия

✓

наименование предприятия

- в области фундаментальных и
поисковых научных исследований

указать область исследований

2 Результаты ВКР:

- рекомендованы к опубликованию
- рекомендованы к внедрению
- внедрены

указать где

указать где

акт внедрения

3 ВКР имеет практическую ценность

✓

*Мобильное приложение для учёта задач и
привычек с геймификацией*

в чем заключается практическая ценность

4 Использование ЭВМ при
выполнении ВКР:
(ПО, компьютерное моделирование,
компьютерная обработка данных и др.)

✓

Android Studio, Firebase

Наличие электронной версии ВКР с

% оригинальности

Студент

Должность

Подпись

Дата

Инициалы Фамилия

РПИС-11

М. Л. Ларкин

Группа

Подпись

Дата

Инициалы Фамилия

Руководитель
ВКР

ст. препод.

Е.В. Расеева

Должность

Уч.степень, звание

Подпись

Дата

Инициалы Фамилия

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Федеральное государственное бюджетное образовательное учреждение
высшего образования**
«Поволжский государственный университет телекоммуникаций и информатики»

РЕФЕРАТ

Название	Разработка мобильного приложения для улучшения продуктивности и качества жизни.
Автор	Ларкин Максим Леонидович
Руководитель ВКР	Расеева Екатерина Викторовна
Ключевые слова	Kotlin, Room, XML, Firebase, Google Sign-in
Дата публикации	2025
Библиографическое описание	Ларкин, М.Л. Разработка мобильного приложения для улучшения продуктивности и качества жизни [Текст]: бакалаврская работа / М.Л. Ларкин. Поволжский государственный университет телекоммуникации и информатики (ПГУТИ). Факультет кибербезопасности и управления (Ф1). Кафедра программной инженерии (ПрИ): Рук. ВКР - Расеева Е.В. – Самара. 2025. – 85с.
Аннотация	Выпускная квалификационная работа посвящена разработке мобильного приложения Nabochi для повышения продуктивности и формирования полезных привычек. Приложение сочетает трекинг задач, систему опыта, достижения и кастомизацию персонажа на основе геймификации. Для реализации использован стек Android/Kotlin, Room и Firebase. Описаны этапы проектирования, выбор технологий и особенности UX. Разработанное решение отличается простотой, вовлечённостью и подходит для широкой аудитории.

Руководитель ВКР

_____ Е.В. Расеева

Подпись

Дата

Инициалы Фамилия

Введение

Каждый человек в какой-то момент сталкивается с необходимостью упорядочить свою жизнь: записывать важные дела, строить планы, формировать полезные привычки. Сегодня для этого всё чаще используются мобильные приложения — они позволяют заменить бумажные списки, напоминания в мессенджерах и разрозненные заметки одним удобным инструментом. Однако за внешней простотой «списка дел» скрывается сложная задача: помочь человеку не просто записывать, а действительно выполнять намеченные цели.

Большинство популярных приложений для продуктивности предлагают привычный набор функций — задачи, повторы, напоминания, статистику. Но с течением времени пользователи всё чаще теряют мотивацию: интерфейс приедается, прогресс перестаёт быть заметным, а чекбоксы — радовать. Простое выполнение задач не вызывает эмоций, и именно это становится причиной отказа от использования таких приложений.

Одним из возможных решений этой проблемы становится геймификация — добавление игровых элементов в неигровые процессы. Награды, уровни, достижения и визуальный прогресс превращают даже рутинные действия в маленькие победы. Такие элементы повышают вовлечённость, вызывают эмоциональный отклик и формируют позитивную поведенческую петлю: сделал — получил результат — захотел повторить.

В данной работе рассматривается разработка мобильного приложения под названием Nabochi — это Android-приложение, которое объединяет трекер задач и привычек с элементами геймификации и визуального прогресса. Пользователь выполняет задачи, получает опыт (XP), повышает уровень, зарабатывает монеты, восстанавливает жизни, открывает достижения и может кастомизировать персонажа. При этом вся механика направлена на поддержку регулярности и формирования устойчивых привычек в игровой, ненавязчивой форме.

Объектом исследования является процесс формирования и поддержки продуктивного поведения пользователя с помощью цифровых инструментов. Предметом исследования является мобильное приложение, реализующее трекинг задач и привычек с включённой системой геймификации.

Цель бакалаврской работы — разработка Android-приложения, объединяющего функциональность трекера продуктивности с современными игровыми элементами, обеспечивающими вовлечённость и визуальный отклик.

Для достижения цели необходимо решить следующие задачи: провести анализ существующих решений, определить требования к функциональности и интерфейсу, спроектировать архитектуру приложения и модель данных, реализовать ключевые модули с применением современных технологий, провести управленческий анализ и определить перспективы развития проекта.

Во введении обоснована актуальность разработки мобильных приложений для продуктивности, сформулированы цель, задачи, объект и предмет исследования.

Первая глава посвящена анализу существующих подходов к продуктивности и формированию привычек, обзору аналогов, методам геймификации, а также обоснованию архитектурных и технологических решений.

Вторая глава включает проектирование и поэтапную реализацию мобильного приложения “Nabochi”: описаны архитектура, структура базы данных, ключевые модули, пользовательский интерфейс и способы взаимодействия.

Третья глава содержит управленческий анализ проекта: представлены этапы жизненного цикла разработки, SWOT- и GAP-анализы, GROW-модель, матрица МакКинзи и метрики успеха, что позволяет оценить перспективы развития и эффективность созданного приложения.

В заключении сформулированы основные выводы по результатам работы и предложены направления дальнейшего развития приложения.

1 Анализ предметной области и теоретические основы

1.1 История планировщиков, трекеров и продуктивности

Идея самоменеджмента и системного подхода к организации личного времени уходит корнями в древность — ещё в античности философы, такие как Сенека и Марк Аврелий, подчёркивали важность дисциплины и самонаблюдения. Однако в более привычной форме управление задачами начало активно развиваться в XIX–XX веках, когда появились первые попытки формализации деловой активности.

Широкое распространение получили бумажные планировщики, ежедневники, чек-листы и таблицы, используемые как в профессиональной, так и в личной среде. В дальнейшем появились методики тайм-менеджмента, одной из наиболее известных стал подход GTD (Getting Things Done), предложенный Дэвидом Алленом в 2001 году. Он оказал значительное влияние на формирование современных представлений о продуктивности.

С развитием цифровых технологий начался переход от бумаги к экрану: сначала появились программы для ПК, затем — мобильные приложения. Особенно активно данная категория стала развиваться с массовым распространением смартфонов и доступных интернет-сервисов. Сегодня на рынке существует огромное количество решений — от простейших «to-do»-листов до сложных систем трекинга привычек с визуализацией, геймификацией и возможностями аналитики [8].

Мобильные приложения для задач, привычек и самоорганизации заняли стабильное место в повседневной жизни. Они особенно востребованы среди студентов, молодых специалистов и представителей цифровых профессий. Пользователи отмечают, что такие приложения помогают поддерживать режим, формировать полезные привычки и ощущение контроля над собственным временем.

Однако при всей популярности подобных инструментов перед разработчиками стоит важная задача — не просто предоставить функции для

фиксации задач, но и сформировать устойчивую мотивацию пользователя к регулярному взаимодействию с системой.

На практике многие пользователи теряют интерес к подобным приложениям уже через 1–2 недели после установки. Основными причинами этого являются отсутствие немедленного подкрепления за выполненные действия (в отличие от игр или социальных сетей), однообразие интерфейса и сценариев взаимодействия, а также психологическая усталость от постоянного самоконтроля.

В этом контексте становится очевидной необходимость внедрения геймификационных подходов — игровых элементов, направленных на повышение вовлечённости, удержание внимания и формирование положительной поведенческой петли. Такие элементы, как прогресс, уровни, достижения и внутренняя мотивация через визуальный отклик, позволяют превратить взаимодействие с приложением в эмоционально подкреплённый и интересный процесс.

1.2 Геймификация как инструмент повышения вовлечённости

Геймификация (от англ. gamification) — это применение игровых элементов и механик в неигровом контексте для увеличения вовлечённости, мотивации и удержания пользователей. Впервые этот термин был широко зафиксирован в 2010 году, хотя сама идея использовалась задолго до этого [6].

Согласно определению исследователя Себастьяна Детердинга, геймификация — это «использование элементов игрового дизайна в неигровых продуктах и услугах для повышения мотивации и взаимодействия». В отличие от полноценной игры, геймификация не создаёт отдельный игровой мир, а лишь внедряет игровые правила, награды и прогресс в привычные действия — такие как обучение, планирование, работа или привычки.

Основная цель геймификации — превратить рутину в увлекательный процесс, вызвать у пользователя эмоциональную привязку и интерес к

возвращению в приложение, тем самым повышая регулярность использования.

Наиболее часто применяемыми элементами геймификации являются:

Опыт (XP, Experience Points) — начисляется за выполнение задач, создавая ощущение роста.

Уровни (Levels) — отражают прогресс, часто связаны с получением новых «званий», способностей или визуальных изменений.

Награды (Rewards) — могут быть внутренними (очки, значки, ачивки) или внешними (скины, кастомизация, доступ к новым функциям).

Жизни (Lives) — ограничивают количество неудачных попыток или ошибок, создавая эффект «на кону».

Эти механики работают по тем же психологическим принципам, что и в видеоиграх: мотивация через достижение, петля вознаграждения, микроцели, визуальное подтверждение прогресса.

В контексте приложения Nabochi именно такие механики играют ключевую роль: XP и уровни отражают прогресс пользователя, жизни теряются при пропущенных действиях и могут быть восстановлены за монеты, а система ачивок и визуального фидбека усиливает чувство достижений.

Существуют успешные примеры внедрения подобных механик в продуктах, таких как Duolingo, Forest и Habitica. Например, Duolingo — один из самых известных примеров успешной геймификации: здесь реализована прокачка уровней, начисление XP, счётчик непрерывных дней (streak), а также присутствуют визуальные персонажи и награды за активность. В системе есть лиги и соревнования между пользователями, что поддерживает мотивацию.

В приложении Forest пользователь «сажает деревья» за каждый непрерывный отрезок фокусировки, что создает эффект роста и визуального прогресса. Наградой за продуктивность становится целая «плантация», а внутренняя валюта (монеты) позволяет покупать новые виды деревьев.

Habitica же представляет собой одну из самых «игровых» систем трекинга: задачи здесь оформлены как битвы с монстрами, за выполнение которых пользователь получает опыт, золото и различные предметы [9]. Кроме того, в системе реализованы снаряжение, питомцы, уровни и гильдии, что делает процесс достижения целей похожим на полноценную игру.

Эти примеры демонстрируют, насколько эффективно игровые механики могут удерживать внимание пользователя и формировать регулярное взаимодействие.

1.3 Психология мотивации: внутренняя и внешняя

Одной из наиболее признанных концепций мотивации в современной психологии является теория самодетерминации (Self-Determination Theory, SDT), разработанная Эдвардом Деси и Ричардом Райаном. Эта теория разделяет мотивацию на внутреннюю и внешнюю, и утверждает, что устойчивая и глубинная мотивация формируется только изнутри, когда человек чувствует:

- автономию — он сам выбирает, что делать;
- компетентность — он чувствует, что справляется;
- связанность — он ощущает участие в чём-то важном.

Во внешней мотивации человек действует ради вознаграждения или избегания наказания (например, чтобы получить оценку, похвалу, деньги). Она может работать, но редко приводит к долгосрочным изменениям в поведении.

Внутренняя мотивация — это то, что побуждает нас делать что-то ради самого процесса: изучать, пробовать, достигать. Приложения, которые строят взаимодействие только на «очки + награда» — часто теряют пользователя со временем. Именно поэтому важно сочетать геймификацию с психологически обоснованным подходом к мотивации.

Формирование привычек — это не просто повторение действий, а процесс, связанный с конкретным циклом, который описал Чарльз Дахигг в книге «Сила привычки». Этот цикл называется *habit loop* (петля привычки) и включает три ключевых элемента: триггер (*cue*) — это событие или контекст, запускающий поведение, например, уведомление от приложения; привычное действие (*routine*) — само поведение, такое как отметка задачи как выполненной; и, наконец, вознаграждение (*reward*) — положительное подкрепление, которым могут выступать XP, значок или хвалебное сообщение.

Эта петля формирует нейронные связи в мозгу, и при повторении поведение закрепляется. Именно её и используют в современных продуктивных приложениях, чтобы помочь пользователю выработать устойчивую модель действий.

Одним из главных вызовов для разработчика является удержание пользователя на длинной дистанции. Для этого существуют различные техники, мотивирующие человека возвращаться в приложение ежедневно. Например, механизм «*streak*» или «цепочка дней» обрывается при пропуске, что вызывает эффект «жалко терять прогресс». Динамический прогресс поддерживается ежедневным пополнением шкалы опыта, ростом или достижением целей. Дополнительно можно внедрять ограниченные по времени награды, например, предоставлять доступ к особой функции или скину только за семь дней подряд. Персональные уведомления с обращением к имени пользователя, упоминанием текущей цели и похвалой также способствуют возвращаемости.

Такие элементы формируют эмоциональную привязку, легкую зависимость от процесса и ощущение контроля над своей жизнью.

Приложение *Nabochi* проектируется с учетом этих принципов: пользователь чувствует автономию благодаря возможности выбирать привычки и кастомизировать приложение; получает разнообразные вознаграждения, такие как XP, звания, монеты и ачивки; выстраивает петлю

— день переходит в задачу, задача приносит XP, XP приводит к повышению уровня, а уровень формирует мотивацию. Кроме того, у пользователя есть стимул заходить каждый день, поскольку в противном случае теряются жизни, streak и прогресс.

1.4 Обзор и сравнение аналогов

Современный рынок мобильных приложений для самоорганизации предлагает множество решений. Среди них можно выделить четыре наиболее популярных и функционально насыщенных продукта, каждый из которых по-своему реализует идеи планирования, привычек и геймификации.

1. Habitica — одно из самых известных приложений, использующих ролевой игровой стиль. Оно превращает повседневные задачи в битвы и квесты, а пользователя — в героя с уровнем, снаряжением и питомцами. За выполнение задач начисляются опыт и золото, а за пропуски персонаж теряет здоровье. Среди преимуществ Habitica стоит отметить выраженную геймификацию, визуальный прогресс и возможность командных миссий. К недостаткам относятся перегруженность интерфейса и достаточно высокий порог вхождения без подробного руководства.

2. Todoist — мощный task-менеджер с минималистичным дизайном, который позволяет организовывать задачи по проектам, меткам и приоритетам. В приложении присутствует базовая статистика продуктивности (очки «кармы»), однако полноценная геймификация отсутствует. Его основные достоинства заключаются в простоте и наличии мощной системы фильтров. Среди минусов можно выделить отсутствие поддержки привычек и слабую визуальную обратную связь.

3. Productive ориентирован именно на формирование привычек, предоставляя возможность отслеживать прогресс и просматривать его в виде графиков. Интерфейс визуально приятный, в приложении есть напоминания и мотивирующие цитаты, однако игровые элементы отсутствуют. Среди плюсов выделяются стильный интерфейс и удобный пользовательский опыт, а вот

платная подписка требуется для большинства функций, и глубокой геймификации здесь нет.

4. Fabulous сочетает привычки, таймеры, образовательные модули и психологические приёмы. Приложение отличается визуальной и эмоциональной насыщенностью, но в большей степени ориентировано на сопровождение пользователя по заранее заданным «путям», чем на гибкую кастомизацию. К достоинствам можно отнести атмосферность и выраженный мотивационный эффект, однако свобода пользователя ограничена, а сам подход сильно уклоняется в сторону лайф-коучинга.

При просмотре итоговых результатов сравнения функциональности приложений в табл. 1.1 можно отметить, что ни одно из популярных приложений не объединяет одновременно полноценный трекинг и привычек, и задач в едином UX-потоке, а также гибкую систему визуального прогресса с использованием XP, streak и достижений. Кроме того, ни одно приложение не предлагает одновременно продуманную и наглядную аналитику в виде графиков и визуальные, эмоциональные элементы без перегруза игровыми сценариями.

Таблица 1.1

Сравнение аналогов

Характеристика	Habitica	Todoist	Productive	Fabulous	Habochi
Задачи	Да	Да	Только привычки	Да	Да
Привычки	Да	Нет	Да	Да	Да
Геймификация	Да	Не совсем	Нет	Недостаточная	Да
Визуальный персонаж	Да	Нет	Нет	Нет	Да
Магазин / награды	Да	Нет	Нет	Нет	Да
Графики и статистика	Слишком базовые	Да	Да	Слишком базовые	Да
Баланс «игра–инструмент»	Сильный перекос в RPG	Да	Более серьезное	Более серьезное	Да

Nabochi стремится занять среднюю позицию между инструментом продуктивности и мотивирующей игрой. В отличие от Habitica, он не уводит пользователя в RPG-реальность, а сохраняет фокус на реальных целях. В отличие от классических task-менеджеров — предлагает эмоциональный отклик и визуальную прогрессию.

Таким образом, Nabochi позиционируется как лёгкое, визуально приятное и функционально сбалансированное решение, ориентированное на пользователей, которым важно не только выполнить, но и почувствовать прогресс.

1.5 Архитектура Android-приложений и UX-подходы

Разработка мобильных приложений требует чёткой архитектурной структуры, которая упрощает поддержку, масштабирование и тестирование, а также обеспечивает гибкость при добавлении нового функционала [1]. Архитектура определяет способ организации кода и взаимодействия между слоями приложения, что напрямую влияет на надёжность, читаемость и дальнейшую сопровождаемость проекта. В условиях роста требований к качеству пользовательского опыта и скорости разработки, выбор подходящей архитектурной модели становится особенно важным. За годы развития Android-разработки сформировались три основных подхода к архитектуре:

1. MVC (Model–View–Controller) — один из старейших архитектурных шаблонов. В нём компонент Model отвечает за хранение и обработку данных, View — за отображение информации пользователю, а Controller реализует логику взаимодействия между моделью и представлением. Однако в Android роль контроллера часто оказывается размыта: активности и фрагменты берут на себя лишнюю логику, из-за чего усложняется сопровождение и тестирование кода.

2. MVP (Model–View–Presenter) — архитектурный шаблон, в котором вся логика и взаимодействие между моделью и представлением сосредоточены в компоненте Presenter. Компонент View в данном случае

выполняет роль «простого» отображателя, не содержащего логики. Такой подход повышает тестируемость и делает код более структурированным. Однако он требует ручного связывания компонентов и может быть сложным в реализации при работе с жизненным циклом Activity и Fragment.

3. MVVM (Model–View–ViewModel) — архитектурный подход, официально рекомендуемый Google, особенно в сочетании с библиотеками Jetpack. В этой модели компонент ViewModel управляет состоянием интерфейса и не содержит прямых ссылок на View, что обеспечивает слабую связанность компонентов. Использование LiveData или StateFlow делает интерфейс реактивным — UI автоматически обновляется при изменении данных [10]. Дополнительно поддерживается Data Binding, что позволяет сократить связность кода и облегчить обновление данных в представлении.

Основные различия между архитектурными паттернами MVC, MVP и MVVM с точки зрения удобства реализации, поддержки и применения в Android приведены в табл. 1.2.

Таблица 1.2

Сравнение архитектурных паттернов

Критерий	MVC	MVP	MVVM
Простота внедрения	Простая	Средняя сложность	Требуется Jetpack/опыт
Разделение обязанностей	Слабое	Хорошее	Отличное
Тестируемость	Ограниченная	Отличная	Отличная
Использование в Android	Устарело	Распространено	Рекомендуется Google
Связь с жизненным циклом	Не учитывает	Сложности	Поддерживается ViewModel
Поддержка Jetpack	Нет	Частично	Полная

Проект Nabochi построен на архитектуре MVVM, потому что ViewModel отлично отделяет бизнес-логику от UI, упрощая логику экрана. Состояния экрана, такие как жизни, уровень, XP и привычки, можно безопасно хранить и

восстанавливать при поворотах экрана и других событиях жизненного цикла. Использование LiveData или MutableStateFlow значительно упрощает обновление пользовательского интерфейса. Кроме того, эта архитектура позволяет подключать Room и DataStore напрямую к ViewModel через слой Repository, а бизнес-логику легко тестировать без зависимости от Android-фреймворка.

Современные приложения, особенно трекеры привычек, требуют не только архитектурной чистоты, но и внимания к UX (User Experience). Пользовательский опыт — это то, что удерживает внимание, формирует привязанность и превращает действие в рутину.

В проекте Nabochi реализованы следующие UX-принципы: доступ к привычке или задаче осуществляется за 1–2 действия, что минимизирует количество кликов; все изменения, такие как начисление XP, изменение количества жизней или прогресса, отображаются мгновенно, обеспечивая визуальный отклик. Интерфейс приложения построен с учётом эмоциональной составляющей: используются персонажи, ачивки и мягкие анимации. Кроме того, поддерживается ежедневный ритуал — привычки и задачи можно задавать с датами и повторами, а система наград формирует у пользователя ощущение «возврата» за совершённые действия.

Плавный пользовательский опыт в связке с реактивной архитектурой MVVM позволяет сделать приложение стабильным, отзывчивым и удобным для повседневного использования.

1.6 Обоснование технологий и библиотек

При проектировании мобильного приложения Nabochi особое внимание было уделено выбору инструментов и технологий, обеспечивающих надёжность, масштабируемость и современный пользовательский опыт. Используемый стек был подобран с учётом требований платформы Android, архитектурных подходов, удобства поддержки и возможностей дальнейшего расширения.

В качестве основного языка программирования был выбран Kotlin — официальный язык разработки под Android, поддерживаемый Google [5]. Его преимущества включают лаконичный синтаксис, встроенную защиту от ошибок, связанных с null-значениями, а также нативную поддержку корутин для организации асинхронной логики. Kotlin отлично интегрируется с Android Jetpack, что делает его идеальным выбором для современных Android-проектов.

Для построения пользовательского интерфейса была использована стандартная View-система Android в связке с разметкой XML. Такой подход обеспечивает точный контроль над расположением элементов и широкие возможности кастомизации. Он совместим с большинством библиотек и подходит для реализации анимированных, визуально отзывчивых экранов.

Хранение данных организовано с использованием Room — библиотеки, предоставляющей удобную обёртку над SQLite и полностью интегрированной в архитектуру MVVM [2]. Она позволяет описывать сущности, выполнять запросы через DAO-интерфейсы, поддерживает миграции и гарантирует стабильность хранения данных. Дополнительно применён DataStore, современный инструмент для сохранения простых настроек и параметров пользователя (таких как тема оформления, количество XP, жизней и прочее). В отличие от устаревшего SharedPreferences, DataStore обеспечивает асинхронную работу на основе Kotlin Flow и повышенную надёжность [3].

Для реализации авторизации в приложении используется Firebase Authentication, поддерживающая вход через Google-аккаунт. Это решение обеспечивает пользователю быстрый и безопасный способ входа, а разработчику — простой доступ к данным об аккаунте (имя, email, аватар) и уникальный идентификатор (UID), который в будущем может быть использован для синхронизации данных между устройствами. Firebase был выбран за официальную поддержку, быструю интеграцию и проверенную безопасность [4].

Визуальная часть приложения обогащена анимациями и графическими компонентами. Для отображения статистики используется библиотека WilliamChart, которая позволяет реализовать диаграммы прогресса (BarChart, LineChart), адаптированные под стилистику приложения. Для оживления интерфейса применяется ViewAnimations — библиотека готовых анимационных эффектов, используемых при появлении достижений, смене экранов и выполнении действий. В проекте также используются VectorDrawable для масштабируемых иконок и изображений, Material Components для реализации кнопок, чекбоксов и прогресс-баров с поддержкой системных тем, а также Google Fonts, которые позволяют подключать пользовательские шрифты и делают интерфейс более выразительным.

Таким образом, выбранный стек технологий в проекте Nabochi обеспечивает надёжное и безопасное хранение данных, стабильную и удобную авторизацию, визуально привлекательный и отзывчивый интерфейс, а также открывает возможности для масштабирования и дальнейшего развития проекта.

1.7 Выводы по главе

Проведённый анализ показал, что современные мобильные приложения для продуктивности и формирования привычек нередко сталкиваются с рядом типичных проблем. Во-первых, отмечается недостаточная удерживающая способность: интерес пользователя часто снижается уже через 1–2 недели использования. Во-вторых, многим решениям не хватает комплексного подхода к мотивации — они опираются либо на внешнее подкрепление, либо на базовую геймификацию, не учитывая психологические закономерности формирования устойчивого поведения. Также часто наблюдается разделение функциональности: приложения фокусируются либо только на задачах, либо только на привычках, либо на лайфстайле, не объединяя эти аспекты в единую систему. Помимо этого, распространён низкий уровень визуального отклика и эмоционального взаимодействия: отсутствуют анимации, игровые элементы,

система достижений или кастомизация интерфейса. Ещё одной проблемой становится нехватка механик прогресса — таких как внутренняя валюта, уровни и персонажи, которые способны формировать у пользователя мотивационную привязанность.

Разрабатываемое приложение Nabochi ориентировано на решение указанных проблем за счёт интеграции геймификационных механизмов, психологически обоснованного подхода к мотивации и современного архитектурного стека. Совмещение трекинга задач и привычек превращает приложение в универсальный инструмент самоорганизации. Система XP, жизней, достижений и уровней обеспечивает разнообразие как внутренних, так и внешних стимулов. Игровая подача интерфейса создаёт эмоциональную вовлечённость и ощущение личного прогресса. Архитектура на основе MVVM с использованием современных технологий, таких как Room, DataStore и Firebase, способствует масштабируемости и удобству поддержки.

Таким образом, Nabochi решает не только техническую, но и поведенческую задачу — формирует у пользователя устойчивую модель взаимодействия с системой продуктивности, основанную на вовлечении, визуальном отклике и системном подкреплении повседневных действий.

2 Проектирование и реализация мобильного приложения “Nabochi”

2.1 Постановка задачи

Целью проекта является разработка мобильного Android-приложения Nabochi, сочетающего в себе:

- трекинг задач и привычек;
- систему геймификации с XP, уровнями, достижениями и жизнями;
- персонализацию, аналитику и визуальную обратную связь;
- сохранение данных с возможностью авторизации через Google-аккаунт.

Приложение должно мотивировать пользователя к регулярному выполнению задач и формированию полезных привычек за счёт игровых элементов, а также быть технически стабильным, визуально привлекательным и удобным в использовании.

В ходе выполнения проекта необходимо решить следующие задачи:

1. Проанализировать предметную область и существующие решения.
2. Определить пользовательские потребности и боли.
3. Сформулировать функциональные и нефункциональные требования.
4. Разработать архитектуру приложения на основе MVVM.
5. Реализовать модули, включающие трекинг привычек и задач, систему XP и уровней, управление жизнями, магазином и достижениями, а также реализовать авторизацию и модуль статистики.
6. Обеспечить удобный UX, стилизацию и плавную анимацию.
7. Провести ручное тестирование ключевых сценариев.

Для оформления требований к функциональности приложения была составлена табл. 2.1, в которой сгруппированы ключевые модули и ожидаемое поведение.

Таблица 2.1

Функциональные требования

Категория	Требование
Задачи и привычки	Создание, редактирование, повторяемость, выполнение, учёт за 3 дня
ХР и уровни	Формула уровня, отображение прогресса, звания
Жизни и магазин	Потеря жизней за пропуски, восстановление за монеты, кнопка аптечки
Ачивки	Прогресс, разбивка на полученные/неполученные
Авторизация	Вход через Google, загрузка аватарки и имени
Хранение	Локально через Room + DataStore
Статистика	Графики выполнения, стилизация, сравнение по дням
Интерфейс	Тёмная тема, анимации

В дополнение к функциональным, в табл. 2.2 представлены нефункциональные требования, обеспечивающие стабильность, удобство и безопасность приложения.

Таблица 2.2

Нефункциональные требования

Тип	Требование
Производительность	Быстрая загрузка, отзывчивый UI, плавные переходы
Удобство использования (UX)	Простота действий, минимум кликов, понятные иконки
Надёжность	Отсутствие критичных сбоев, устойчивость при поворотах экрана и перезапусках
Модульность	Возможность расширения функционала (например, онлайн-синхронизация)
Безопасность	Использование проверенных API (Firebase), работа только с нужными разрешениями

Этапы проектирования и реализации приложения были разбиты по времени и представлены в виде Gantt-диаграммы, которая позволяет наглядно отследить хронологию разработки. Диаграмма представлена на рис. 2.1.

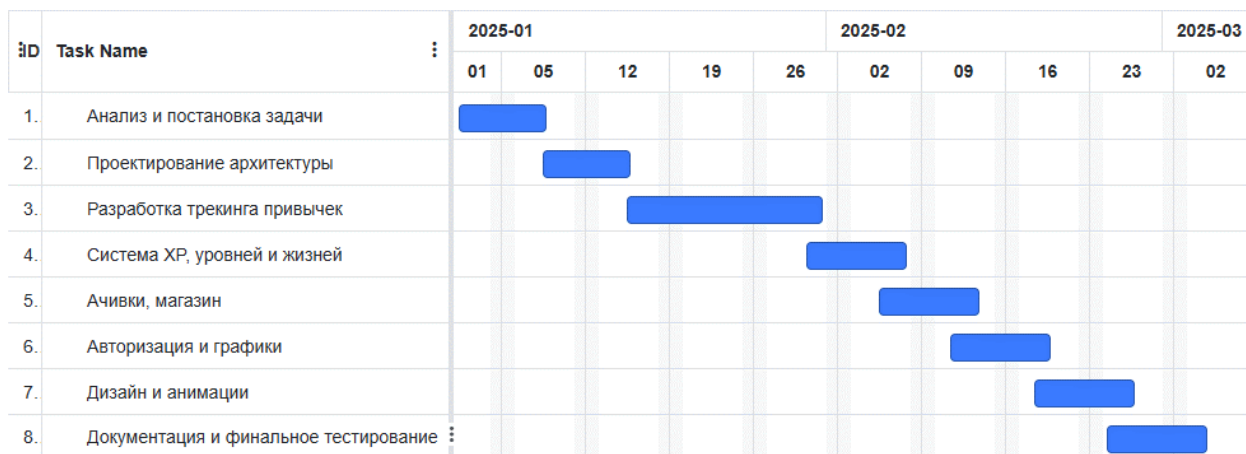


Рис. 2.1 - Gantt-диаграмма разработки

Для систематизации целей разработки проекта была использована диаграмма целей (Goal Breakdown). На рис. 2.2 показано разбиение общей цели на подцели.



Рис. 2.2 - диаграмма целей (Goal Breakdown)

Таким образом, постановка задачи в проекте Nabochi включает как технические, так и поведенческие аспекты. Проект направлен не только на реализацию функциональности, но и на создание эмоционально вовлекающего интерфейса, способствующего формированию привычек и поддержанию продуктивности пользователя.

2.2 Архитектура приложения

Приложение Nabochi построено на архитектурном шаблоне MVVM (Model–View–ViewModel), который является официально рекомендуемым Google подходом для Android-приложений. Эта архитектура обеспечивает разделение логики и пользовательского интерфейса, устойчивость к изменениям жизненного цикла, повышенную тестируемость, а также удобную интеграцию с библиотеками Jetpack.

MVVM особенно эффективно работает в связке с LiveData / StateFlow, ViewModel, Room, и DataStore, что делает его идеальным выбором для проекта, где важны реактивность интерфейса и устойчивость при поворотах/перезапуске.

Ключевой принцип архитектуры MVVM заключается в отделении бизнес-логики от представления. View (Fragment или Activity) отвечает за отображение данных и передачу действий пользователя. ViewModel хранит состояние пользовательского интерфейса, обрабатывает логику взаимодействия с данными и предоставляет их во View через LiveData. Repository, в свою очередь, служит посредником между ViewModel и источниками данных, такими как Room, DataStore и Firebase.

Пример потока данных:

1. Пользователь нажимает на CheckBox (id=checkBox) в элементе item_task.xml.

2. Это вызывает обработчик внутри TaskAdapter.TaskViewHolder:

```
cbCompleted.setOnCheckedChangeListener { _, isChecked ->
    onChecked(task, isChecked)
}
```

3. Вызывается внешняя лямбда onChecked(task, isChecked) — она передаётся извне в TaskAdapter через конструктор.

4. Внешняя логика (например, во ViewModel или Fragment) реагирует и обновляет состояние задачи в базе (Room), что вызывает обновление UI через LiveData.

Структура архитектуры MVVM в проекте Nabochi реализована в виде чётких слоёв взаимодействия: пользовательский интерфейс, слой логики (ViewModel), репозитории и источники данных. Их взаимодействие представлено на рис. 2.3.

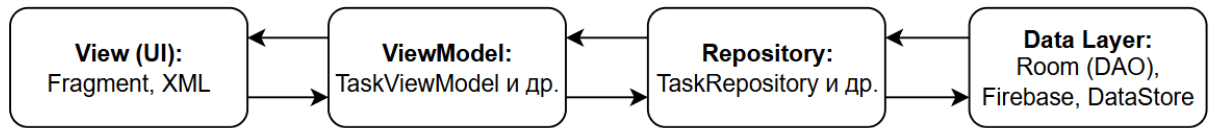


Рис. 2.3 — Архитектурные слои MVVM

Каждый слой архитектуры обладает единственной зоной ответственности (принцип Single Responsibility Principle), построен по принципу высокой когезии и слабой связанности, а также легко расширяется и тестируется.

В рамках проектирования архитектуры приложения также была составлена UML-диаграмма на примере работы экрана «привычек», отражающая взаимодействие фрагментов, ViewModel и источников данных. Диаграмма представлена на рис. 2.4.

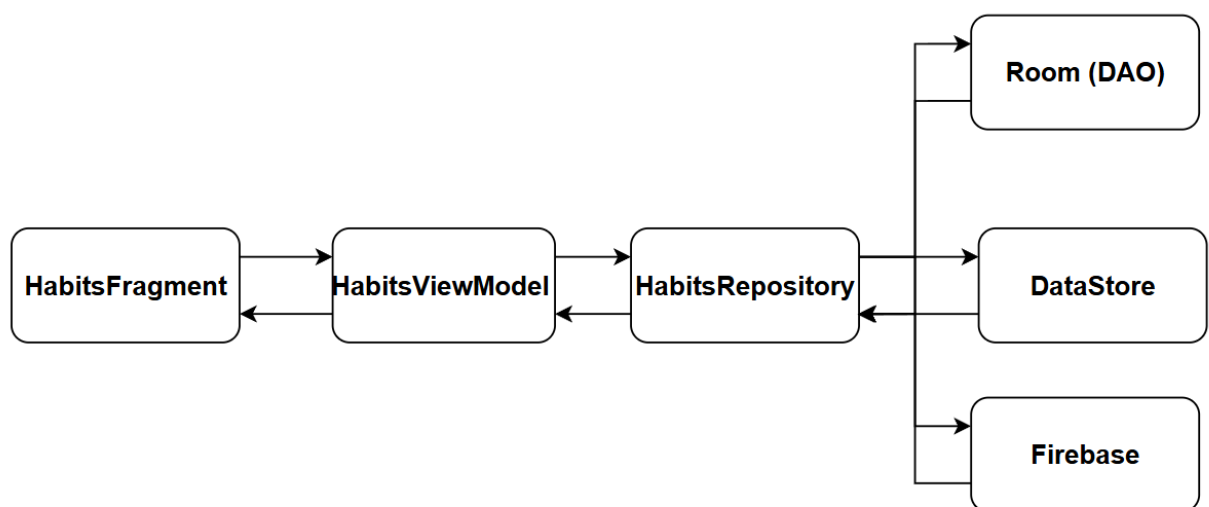


Рис. 2.4 — UML-диаграмма компонентов

Основные ViewModel в проекте разделены по зонам ответственности. Каждая из них обрабатывает своё направление данных и взаимодействует с соответствующими репозиториями. Перечень ViewModel и их назначение приведён в табл. 2.3.

Таблица 2.3

Основные ViewModel и зона ответственности

ViewModel	Назначение
CalendarViewModel	Управление задачами в календарном экране
GamesViewModel	Обработка игровой логики (покупки, опыт, жизни)
HabitsViewModel	CRUD-операции над привычками
MainViewModel	Подготовка данных домашнего экрана: состояние пользователя (опыт, монеты и достижения)
TasksViewModel	CRUD-операции над задачами

Каждая ViewModel подписана на LiveData/StateFlow и обновляет UI при изменениях данных.

Использование MVVM позволило выстроить логически чистую, масштабируемую архитектуру. Благодаря чёткому разграничению слоёв, приложение легко поддерживается и может быть расширено в будущем — например, добавлением облачного синхронизатора, новых экранов, push-уведомлений и т.д.

2.3 Модель данных и база данных

В приложении Nabochi используется связка двух инструментов для локального хранения данных: Room, являющийся библиотекой Android Jetpack и реализующий объектно-реляционную обёртку над SQLite, а также Preferences DataStore — механизм для хранения простых конфигурационных значений, который приходит на смену устаревшему SharedPreferences. Такое распределение позволяет хранить структурированные игровые данные, такие как задачи, привычки, достижения и данные пользователя, в базе данных Room, а интерфейсные и поведенческие настройки — например, тему, дату последнего входа или email — сохранять в DataStore.

Room автоматически генерирует структуру таблиц на основе классов данных, помеченных аннотациями `@Entity`. Каждая сущность представляет собой Kotlin-класс с набором полей и аннотаций (`@PrimaryKey`, `@ColumnInfo` и др.), определяющих структуру таблицы.

В табл. 2.4 представлены основные сущности их назначения и поля.

Таблица 2.4

Основные сущности базы данных Room

Сущность	Назначение	Основные поля
UserEntity	Данные пользователя (локально)	id, xp, level, coins, lives, rank
TaskEntity	Ежедневные/одноразовые задачи	id, title, date, time, isCompleted, importance
HabitsEntity	Привычки с повторами	id, title, isCompleted, iconResId, color, repeatType, repeatDays, startDate
AchievementEntity	Достижения и их состояние	id, title, description, isUnlocked, unlockDate
HabitBonusEntity	Фиксация бонусов за выполнение привычек	id, habitId, weekStart, weekEnd

Компоненты Room позволяют реализовать слабые связи между таблицами через логику приложения и обращения к `userId`. Один пользователь (UserEntity) является владельцем всех задач, привычек и достижений, а выполнение TaskEntity и HabitEntity влияет на характеристики пользователя, такие как опыт, уровень и количество жизней. Сущность AchievementEntity логически привязана к событиям в других таблицах, но не содержит внешних ключей напрямую. Проект не использует ручное проектирование схемы базы данных или SQL-создание таблиц: Room самостоятельно формирует структуру таблиц на основе аннотированных классов, обрабатывает миграции, индексирование и ограничения, а также предоставляет автоматическую компиляцию SQL-запросов и удобную обёртку для взаимодействия с базой.

Таким образом, необходимость в явной ER-диаграмме отпадает, так как архитектура данных читается из самих Kotlin-моделей.

Для получения задач на определённую дату используется следующий запрос (в логике DAO-интерфейса Room):

```
@Query("SELECT * FROM tasks WHERE date = :date")
suspend fun getTasksByDate(date: String): List<TaskEntity>
```

Для хранения небольших параметров, не требующих структуры, таких как тема оформления, email или дата последнего запуска, используется Preferences DataStore. Он позволяет сохранять настройки в формате ключ–значение, использовать асинхронное чтение и запись данных, а также эффективно управлять состоянием приложения без необходимости обращаться к Room.

Использование связки Room + DataStore обеспечивает устойчивое, гибкое и полностью локальное хранение пользовательских данных. Room реализует основную бизнес-логику, а DataStore — быстрое и лёгкое управление настройками. Выбранная структура соответствует архитектуре MVVM и обеспечивает масштабируемость в будущем: от подключения облачной синхронизации до анализа статистики.

2.4 Реализация трекинга привычек и задач

Механизм трекинга задач и привычек является основой пользовательского опыта в приложении Nabochi. Он направлен на формирование устойчивого взаимодействия пользователя с системой и основан на принципах:

- минимального количества действий;
- наглядного отображения прогресса;
- поведенческой мотивации через игровые элементы.

В приложении реализованы два типа трекабельных сущностей:

1. Задачи (Tasks) создаются с указанием названия, описания, даты выполнения, времени и важности. Они поддерживают только одноразовое выполнение, в отличие от привычек, и содержат поле isCompleted, которое определяет факт выполнения задачи.

2. Привычки (Habits) создаются с указанием названия, цвета, иконки и типа повторения (например, ежедневно или по дням недели). Повторяемость настраивается через поля `repeatType` и `repeatDays`. Факт выполнения привычки в конкретную дату не хранится напрямую в самой привычке — для этого используется отдельная таблица `habit_completion`. Каждая запись в сущности `HabitCompletionEntity` фиксирует идентификатор привычки (`habitId`), дату, факт выполнения (`isCompleted`), а также отмечает, был ли начислен бонус за streak (`bonusAwarded`).

На рис. 2.5 – 2.8 представлен основной функционал экрана управления привычками, позволяющий просматривать и редактировать текущие привычки пользователя.

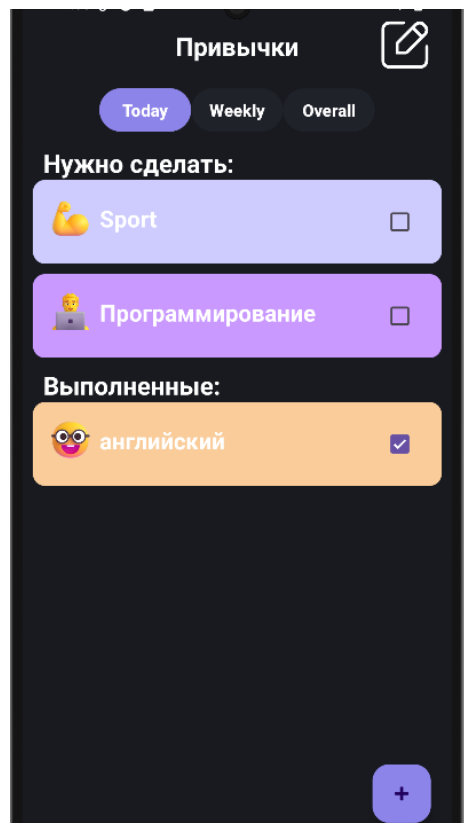


Рис. 2.5 — Привычки на сегодня

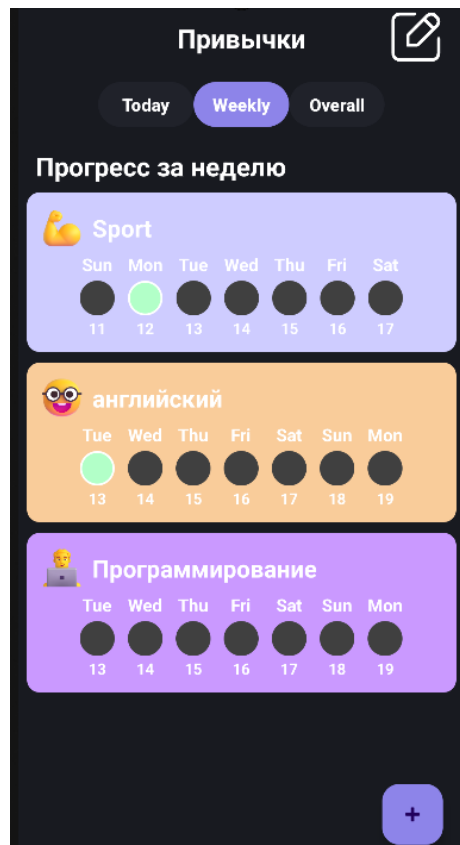


Рис. 2.6 — Привычки на неделю

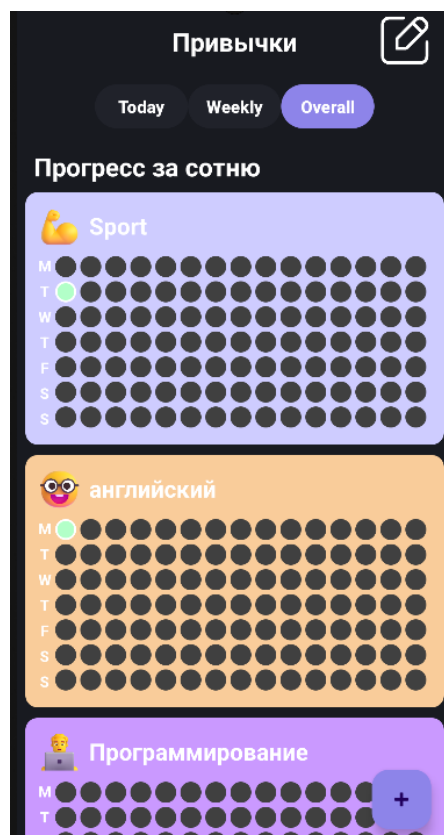


Рис. 2.7 — 100-дневный трекер привычек

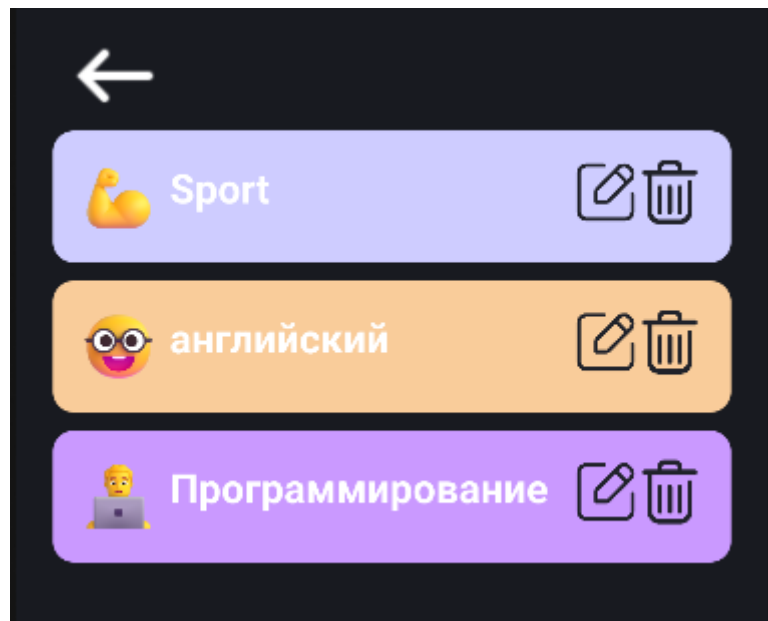


Рис. 2.8 — Экран редактирования привычек

Для создания новой привычки или задачи в приложении предусмотрены отдельные экраны, которые представлены на рис. 2.9 – 2.11.

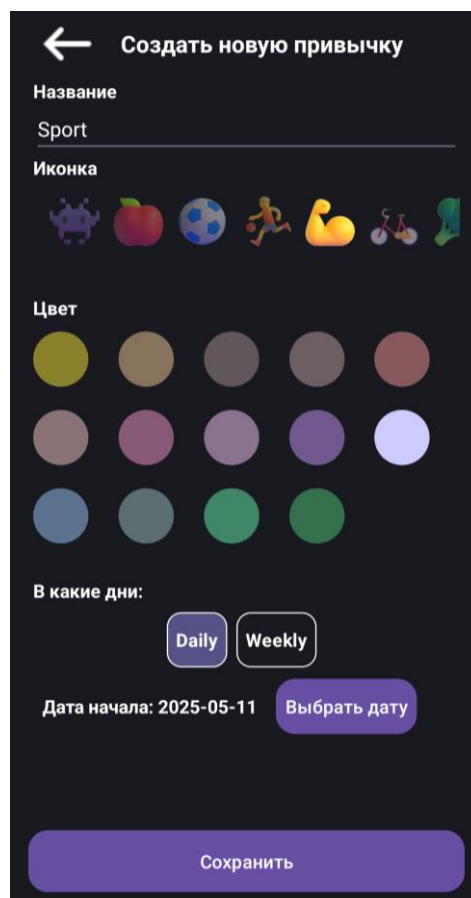


Рис. 2.9 — Экран добавления новой ежедневной привычки

Рис. 2.10 — Добавления привычки за конкретные дни недели

Рис. 2.11 — Экран добавления новой задачи

Для поддержания честности и регулярности взаимодействия в приложении реализовано ограничение: пользователь может отмечать выполнение только за сегодняшний и два предыдущих дня. Это предотвращает массовую отметку задним числом, стимулирует регулярное использование приложения и делает систему ХР и жизнью значимой и защищённой от злоупотреблений.

Проверка реализуется через сравнение выбранной даты с текущей, и при превышении лимита — возможность отметки блокируется.

Пример уведомления, если пользователь попытается отметить выполнение привычки за не наступившую дату приведён на рис. 2.12.

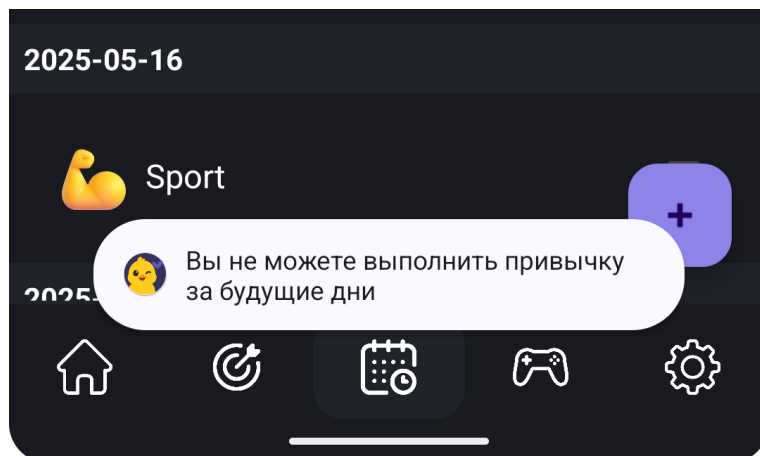


Рис. 2.12 — Уведомление об отмене выполнения привычки

Процесс выполнения задачи или привычки реализован через взаимодействие между элементом интерфейса, адаптером и соответствующей ViewModel. Основные этапы включают следующий порядок: при нажатии на чекбокс в адаптере (TaskAdapter) срабатывает функция-обработчик, которая вызывает переданную лямбду — `onTaskChecked(task, isChecked)` для задач или `onHabitChecked(habitId, isChecked, date)` для привычек. Затем вызывается метод во ViewModel: для задач — `TasksViewModel.toggleCompletion(taskId, isCompleted)`, а для привычек — `HabitsViewModel.toggleHabitCompletion(habitId, isCompleted)`. После этого ViewModel передаёт управление соответствующему репозиторию (TaskRepository или HabitsRepository), где выполняется обновление состояния `isCompleted` в базе данных. В результате задача или привычка помечается как выполненная, начисляются очки опыта (XP), проверяются условия достижения streak и разблокировки ачивок, а пользовательский интерфейс обновляется за счёт реактивных потоков StateFlow.

Потеря жизней реализована следующим образом: при запуске приложения или по расписанию происходит проверка выполнения задач и привычек за предыдущие дни (в пределах двух суток). Если обнаруживаются

пропуски, количество жизней уменьшается, что визуально отражается в виде гашения и анимации иконок в интерфейсе.

Восстановление streak и ХР возможно, если пользователь отмечает выполнение пропущенных действий в течение трёхдневного допустимого окна. В таком случае прогресс восстанавливается автоматически, а визуальные индикаторы streak и ХР обновляются.

В приложении реализовано несколько ключевых экранов (фрагментов), каждый из которых отвечает за свою часть пользовательского взаимодействия. Навигация между ними осуществляется через нижнее меню. Назначение и функции каждого экрана представлены в табл. 2.5.

Таблица 2.5

Взаимодействие пользователя с фрагментами приложения

Экран	Назначение
HomeFragment	Обзор текущего прогресса: ХР, жизни, графики выполнения, приветствие пользователя
HabitsFragment	Отображение всех привычек, отметка выполнения, смена режима отображения (день/неделя)
TasksFragment	Список задач на выбранную дату, сортировка по важности, выполнение/удаление задач
CalendarFragment	Навигация по дням месяца, отображение активности, выбор даты для трекинга
GamesFragment	Экран персонажа: уровень, скины, количество жизней, магазин и достижения
ShopFragment	Покупка кастомизации и скинов за монеты, отображение приобретённых предметов
SettingsFragment	Управление темой, вход/выход из аккаунта, отображение email, восстановление прогресса

Приложение реализует ряд микровзаимодействий, направленных на повышение удобства и эмоциональной вовлечённости. При отметке задачи используется анимированный чекбокс, что обеспечивает визуальный отклик. Для всех базовых действий предусмотрена навигация в один–два шага, что соответствует принципам доступного и интуитивного UX-дизайна.

Таким образом, система трекинга задач и привычек в Nabochi обеспечивает регулярность действий, подкрепляя продуктивное поведение через внутриигровые награды и визуальный прогресс. Она исключает недобросовестное использование (например, отметку задним числом), опирается на мотивационные ограничения и гибко масштабируется под будущие сценарии — календарь, расширенную аналитику, персональные цели и адаптивные режимы отображения.

2.5 Система XP и уровней

Система опыта (XP) и уровней в приложении Nabochi служит ключевым элементом игровой мотивации, направленной на формирование положительного подкрепления и поддержание регулярного взаимодействия пользователя с приложением.

Выполняя привычки и задачи, пользователь зарабатывает очки опыта, которые позволяют постепенно повышать уровень и открывать новые звания.

В отличие от динамического расчёта через формулу, в проекте реализована фиксированная таблица порогов XP для достижения каждого следующего уровня. Такой подход обеспечивает полный контроль над темпом прогрессии, позволяет сбалансировать лёгкость старта и сложность достижения высоких уровней, а также делает систему наглядной и предсказуемой для пользователя.

Значения порогов XP для каждого уровня заданы вручную и приведены в табл. 2.6, что позволяет обеспечить точный контроль над темпом прокачки персонажа.

Таблица 2.6

Пороговые значения опыта для достижения уровней

Уровень	Необходимое общее количество опыта
1	0 XP
2	10 XP
3	40 XP
4	80 XP

5	130 XP
6	190 XP
7	260 XP
8	340 XP
9	430 XP
10	520 XP
11	620 XP

Чтобы дополнительно усилить эффект прогресса, каждому диапазону уровней соответствует собственное мотивационное звание. Они представлены в табл. 2.7.

Таблица 2.7

Звания, присваиваемые пользователю в зависимости от уровня

Диапазон уровней	Присваиваемое звание
1–2	Новичок
3–4	Ученик
5–6	Мастер
7–8	Эксперт
9–10	Легенда
11+	Бог продуктивности

Элементы отображения прогресса реализованы следующим образом: прогресс-бар показывает заполнение шкалы опыта до следующего уровня, текущий уровень отображается числом рядом с персонажем, а текущий опыт выводится в формате XP (например, 83 / 100 XP). Дополнительно каждому уровню соответствует звание, которое представлено в виде текстового сопровождения; в будущем предусмотрена возможность визуального украшения, например, с помощью цвета рамки или значка.

Пример отображения прогресса приведен на рис. 2.13.

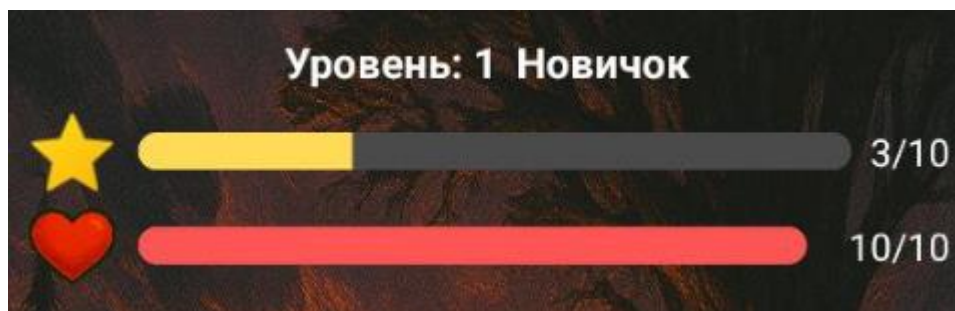


Рис. 2.13 — Отображение прогресса

Достижение нового уровня сопровождается всплывающим уведомлением о повышении и присвоении звания. Таким образом, система ХР и уровней в Nabochi выполняет несколько ключевых функций: она стимулирует краткосрочную активность пользователя за счёт моментального получения опыта, поддерживает долгосрочную вовлечённость через механизмы уровней и званий, формирует эмоциональную привязанность к процессу развития, а также остаётся гибкой и расширяемой, поскольку при необходимости может быть легко масштабирована за счёт добавления новых уровней, наград и геймификационных элементов.

2.6 Жизни персонажа и магазин

В приложении Nabochi реализована система жизней, которая отражает «состояние» внутриигрового персонажа и служит инструментом мягкого стимулирования пользователя к регулярному выполнению задач и привычек. Подход основан на геймификационной метафоре: пользователь заботится о своём персонаже, сохраняя у него жизни за счёт активности.

Основные принципы работы механики жизней заключаются в следующем: максимальное количество жизней ограничено десятью. За каждую пропущенную задачу или привычку за прошедшие сутки снимается одна жизнь. Потеря жизней происходит один раз в день — либо при первом запуске приложения за сутки, либо по внутреннему расписанию. Если на конкретный день не было заданий, жизни не теряются. В случае потери

пользователь видит визуальную анимацию и информационное сообщение. Когда жизни заканчиваются, происходит "смерть" питомца, как показано на рис. 2.14.

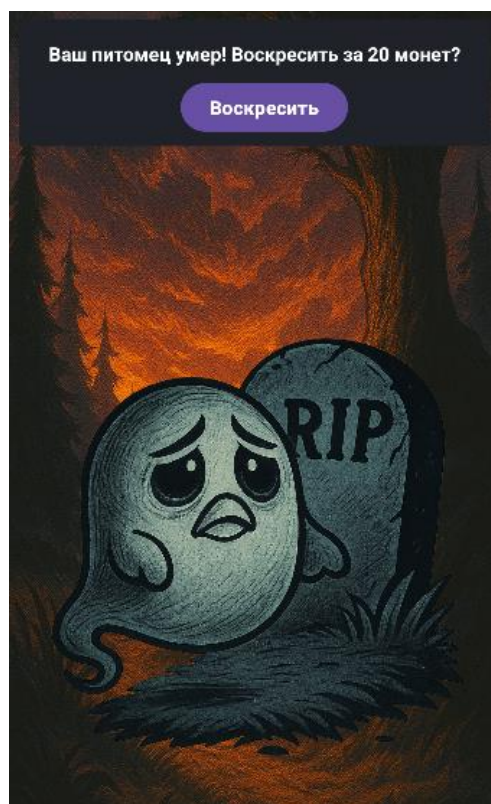


Рис. 2.14 — Смерть питомца

Хранение количества жизней реализовано в сущности `UserEntity` и синхронизируется с интерфейсом через `ViewModel`. Это позволяет мгновенно отображать актуальное состояние персонажа на экране.

Для повышения гибкости предусмотрена возможность восстановить потерянные жизни за монеты, которые зарабатываются в процессе взаимодействия с приложением. Стоимость одной жизни составляет 5 монет. Восстановление возможно только в том случае, если текущее количество жизней меньше десяти. При успешном восстановлении отображается уведомление: «+1 жизнь! Береги себя и продолжай двигаться к целям!» Если условия не выполнены (жизней уже 10 или монет меньше пяти), пользователь получает предупреждение о невозможности действия.

Интерфейс взаимодействия реализован через специальный UI-компонент. Для визуального запуска восстановления жизней используется иконка аптечки (сердечко), которая отображается рядом с жизнями персонажа. При нажатии на иконку появляется окно с подтверждением действия, в котором задаётся вопрос: «Вы действительно хотите потратить 5 монет на восстановление 1 жизни?» Пользователь может выбрать одну из двух кнопок: Да или Отмена. Поведение кнопок программируется с учётом проверок условий. На рис. 2.15 представлен интерфейсный элемент аптечки и всплывающее окно подтверждения восстановления жизни.



Рис. 2.15 — Аптечка и окно подтверждения покупки

Механика жизней в Nabochi усиливает эмоциональную вовлечённость пользователя, поддерживает дисциплину без жёстких ограничений и создаёт основу для расширения игровых функций. Она объединяет стимулирующие элементы и ограничения в единую систему поведенческой обратной связи, формируя у пользователя желание заботиться о персонаже и не пропускать активности.

2.7 Достижения (ачивки)

Система достижений в приложении Nabochi играет роль дополнительной нематериальной мотивации пользователя. Она отмечает важные события и последовательные действия, усиливая чувство прогресса, эмоциональную вовлечённость и удовлетворение от использования приложения.

Достижения в Nabochi не дают прямой выгоды в виде опыта или монет, но выполняют критически важную психологическую функцию. Они поддерживают как краткосрочную, так и долгосрочную мотивацию, формируют эмоциональную привязанность к процессу развития и стимулируют интерес пользователя через элементы коллекционирования. В приложении используется структура сущности AchievementEntity, которая включает уникальный идентификатор достижения (id), название (title), описание условия получения (description), флаг получения (isUnlocked) и дату получения (unlockDate), если достижение уже открыто.

Эта модель позволяет легко управлять состоянием достижений и обновлять интерфейс при изменениях.

Уже реализованные достижения и условия их получения продемонстрированы в табл. 2.7.

Таблица 2.7

Достижения и условия их получения

ID	Название	Условие получения
first_task	Первая задача	Выполнить первую задачу
first_habit	Первая привычка	Выполнить первую привычку
three_in_day	3 за день	Выполнить 3 любых действия за сутки
perfect_day	Идеальный день	Выполнить все задачи за день
seven_streak	Семидневка	7 дней подряд активности
task_master	Мастер дел	Выполнить 50 задач
habit_keeper	Продуктивность — это я	Выполнить 30 привычек
coin_collector	Люблю золото	Заработать 100 монет

Механизм работы системы достижений включает следующие этапы:

1. При выполнении ключевых действий (закрытие streak, выполнение задачи, накопление монет) вызывается автоматическая проверка условий.
2. При совпадении условия, соответствующее достижение переводится в состояние `isUnlocked = true`, с фиксацией даты получения.
3. Пользователь получает визуальное уведомление об открытии новой ачивки.

Раздел достижений в пользовательском интерфейсе разделён на две части: полученные достижения представлены в виде цветных иконок, при этом отображается дата получения и проигрывается анимация при открытии. Неполученные достижения, напротив, отображаются как серые иконки с символом замка, а описание условий раскрывается только после их получения. Кроме того, некоторые достижения могут быть скрыты до момента открытия, что добавляет элемент сюрприза.

Пример полученных и неполученных достижений приведен на рис. 2.16.

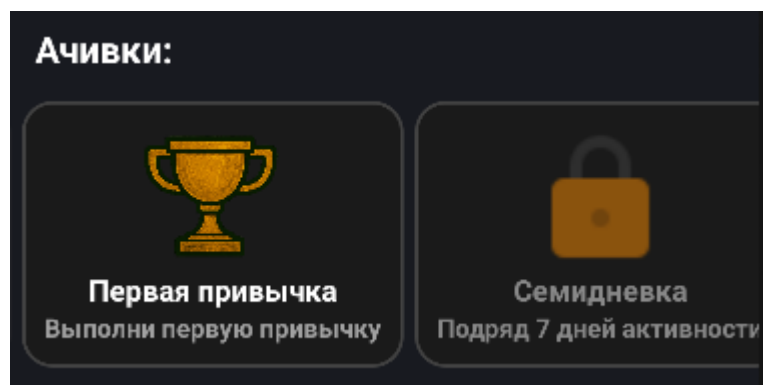


Рис. 2.16 — Полученные и неполученные достижения

Достижения в Nabochi выполняют сразу несколько функций. Они стимулируют краткосрочную мотивацию — например, вызывают желание «закрыть» день или выполнить несколько действий подряд. Кроме того, они формируют долгосрочные цели, такие как стремление собрать все достижения или достичь уникальных условий. Ещё одной важной функцией является

усиление эмоциональной привязанности к приложению, ведь пользователь всегда видит визуальный результат своих усилий.

Эта многоуровневая мотивационная модель делает процесс самоорганизации более интересным и вовлекающим.

В результате, система достижений в Nabochi трансформирует выполнение привычек и задач в цепочку игровых наград, объединяя принципы геймификации с реальной повседневной активностью. Это позволяет пользователю не только эффективно развивать продуктивные привычки, но и получать эмоциональное удовлетворение от видимого роста и успеха.

2.8 Магазин кастомизации

Кастомизация внешнего вида персонажа является важным элементом поддержания эмоциональной привязанности пользователя к приложению. Возможность выбора скинов позволяет усиливать мотивацию через ощущение уникальности и достижения.

Эта механика широко используется в геймифицированных продуктах, таких как Habitica или Duolingo, и направлена на превращение продуктивности в игровой и персонализированный опыт.

В текущей версии Nabochi реализована базовая версия магазина скинов - доступно 5 скинов персонажа (pet_default, pet_flower, pet_knight, pet_wizard, pet_king). Каждый скин имеет свою стоимость в монетах. Скины и их стоимостью приведены в табл. 2.8.

Таблица 2.8

Название и стоимость скинов в магазине

Название скина	Цена (монеты)
Стандартный	0
Цветочный	15
Рыцарь	30
Волшебник	50
Король	100

Подробный процесс и условия покупки скина реализованы следующим образом: покупка становится доступной только при наличии достаточного количества монет у пользователя. В момент покупки происходит списание нужного количества монет и сохранение информации о покупке в локальной базе данных. После приобретения скина пользователь может выбрать его в качестве активного для отображения в интерфейсе. Каждый товар в магазине описывается моделью:

```
data class ShopItem(  
    val id: Int,  
    val name: String,  
    val price: Int,  
    val isPurchased: Boolean,  
    val iconRes: Int  
)
```

Визуальное представление магазина со стандартным и самым дорогим скином изображено на рис. 2.17 - 2.18.



Рис. 2.17 — Магазин со стартовым скином



Рис. 2.18 — Магазин с самым дорогим скином

Информация о купленных товарах хранится локально в базе данных и может быть расширена для синхронизации с аккаунтом в будущем. Пример функционала, реализации и комментарии приведены в табл. 2.9.

Таблица 2.9

Текущее состояние реализации

Функция	Реализация	Комментарий
Интерфейс магазина	Да	Экран отображает список товаров и ценники
Списание монет	Да	При покупке проверяется баланс пользователя
Выбор скина персонажа	Да	Приобретённый скин может быть выбран
Категории товаров	Нет	Пока реализована только категория скинов
Анимации/фоны	Нет	Планируется к реализации в будущих версиях

В будущих версиях планируется расширение функциональности магазина. Разработчики намерены добавить новые категории товаров, такие как фоны, аксессуары и питомцы. В магазине появятся сезонные предметы,

например, новогодние или тематические наборы. Система достижений будет открывать доступ к уникальным скинам, а также будет внедрена система уровней редкости предметов — от обычных до эпических.

Даже на текущем этапе магазин кастомизации в Nabochi выполняет важную геймификационную роль, усиливая мотивацию пользователя и создавая дополнительный стимул к накоплению монет через выполнение привычек и задач. Расширение возможностей кастомизации в дальнейшем позволит ещё больше повысить вовлечённость и эмоциональную привязанность к приложению.

2.9 Авторизация и хранение аккаунта

Для обеспечения персонализации пользователя и подготовки возможности облачного хранения данных в будущем в приложении Nabochi реализована система авторизации через Firebase Authentication. Этот сервис был выбран благодаря поддержке входа через Google в один клик, бесплатности, высокой степени надёжности, автоматической обработке безопасности и обновления токенов, а также полной совместимости с Android SDK и архитектурой MVVM. После успешной авторизации приложение получает уникальный идентификатор пользователя (UID), адрес электронной почты (используется для отображения в профиле), имя пользователя (отображается в интерфейсе) и ссылку на аватар (загружается и кэшируется локально, если имеется).

Эти данные сохраняются локально через DataStore. На данный момент они используются исключительно для отображения профиля внутри приложения и не передаются третьим лицам. UID в перспективе может быть использован для синхронизации данных между устройствами.

Процесс авторизации начинается после нажатия пользователем на кнопку «Войти с Google», что запускает механизм Google Sign-In. Пользователь выбирает нужный аккаунт из списка доступных на устройстве.

Система получает авторизационный токен, передаёт его на серверы Firebase для проверки, и в случае успешной валидации приложение получает основные данные — UID, адрес электронной почты, имя пользователя и ссылку на аватар. Все полученные данные сохраняются локально с помощью DataStore для быстрого отображения в интерфейсе. После этого интерфейс обновляется и отображает имя, аватар и электронную почту пользователя.

Визуальный пример начального экрана с кнопкой входа в аккаунт приведен на рис. 2.19.

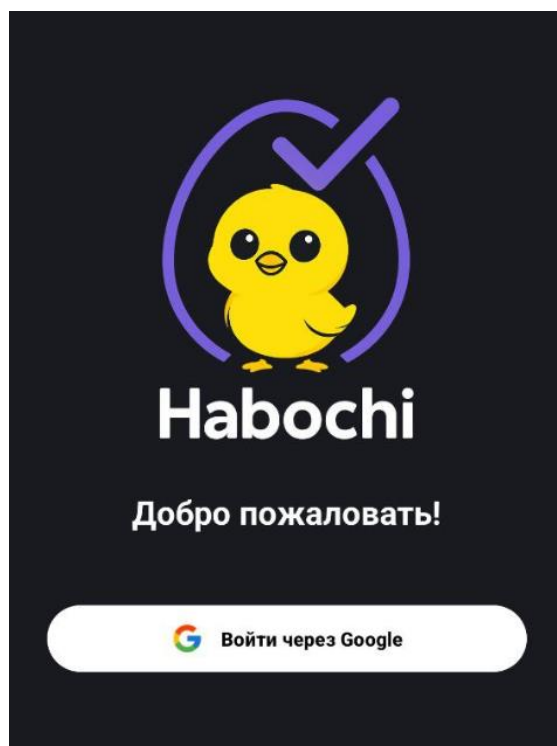


Рис. 2.19 — Экран с кнопкой входа через Google

Функция выхода реализована на экране настроек приложения. При её активации вызывается метод `FirebaseAuth.signOut()`, все данные авторизации (email, UID, имя) удаляются из DataStore, а пользователь возвращается в состояние незарегистрированного доступа, при этом интерфейс адаптируется соответствующим образом.

Для хранения данных приложение использует два подхода. Ключевые сущности — задачи (`TaskEntity`), привычки (`HabitEntity`), достижения (`AchievementEntity`), параметры прогресса (`UserEntity`) — сохраняются в базе

данных Room. Простые параметры, такие как тема оформления, дата последнего входа, а также данные профиля и авторизации (email, имя, UID), хранятся в Preferences DataStore.

Безопасность построена на протоколе OAuth 2.0, что обеспечивает безопасную передачу токенов, автоматическое обновление и проверку подлинности сессии, а также уникальность идентификатора пользователя. Приложение не запрашивает доступ к контактам, геолокации, файлам или другим конфиденциальным данным, что соответствует принципу минимально необходимого доступа (principle of least privilege) и упрощает сертификацию в Google Play.

Реализованная система авторизации сочетает простоту использования и надёжную защиту данных. Она обеспечивает персонализированный опыт, не создавая обязательств для пользователя — функциональность приложения полностью сохраняется и без входа в аккаунт. Такой подход позволяет расширить аудиторию и формирует основу для интеграции облачной синхронизации, резервного копирования и переноса данных в будущих версиях приложения.

2.10 Статистика

Одной из ключевых задач приложения Nabochi является поддержание мотивации пользователя не только через игровые механики, но и посредством наглядного отображения прогресса. Раздел статистики помогает формировать у пользователя ощущение динамики, достижений и контроля над своей продуктивностью.

Графическое представление данных позволяет:

- отслеживать регулярность выполнения задач и привычек;
- замечать тренды в поведении: периоды роста, снижения или стабильности;
- усиливать внутреннюю мотивацию за счёт визуального подтверждения усилий.

Столбчатая диаграмма (BarChart).

Одним из основных графиков, реализованных в приложении, является BarChart — диаграмма, отображающая количество выполненных привычек за каждый из последних 7 дней.

Каждая колонка диаграммы соответствует дню недели и отличается по высоте в зависимости от числа выполнений. Под столбцами указаны сокращения дней недели (Пн, Вт, Ср и т.д.), что делает визуализацию понятной и привычной для пользователя.

Для построения графика используется библиотека WilliamChart, обеспечивающая высокую производительность и гибкую стилизацию элементов.

На рис. 2.20 представлен пример отображения BarChart-графика в приложении.

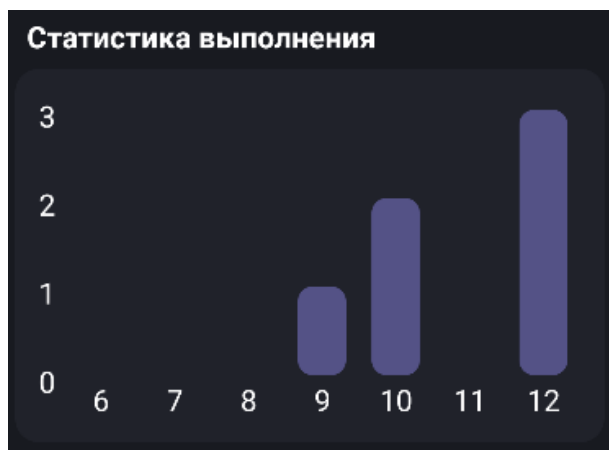


Рис. 2.20 — График выполнения привычек (BarChart)

Линейная диаграмма (LineChart).

Для оценки относительной продуктивности пользователя используется LineChart — график, отображающий процент выполненных действий от общего количества за день. Например, если на понедельник было запланировано 6 задач и привычек, а выполнено только 3, то значение графика составит 50%.

Данный график позволяет отслеживать не только абсолютные цифры, но и эффективность выполнения в динамике, выявлять спады и рост активности.

Пример линейного графика представлен на рис. 2.20.

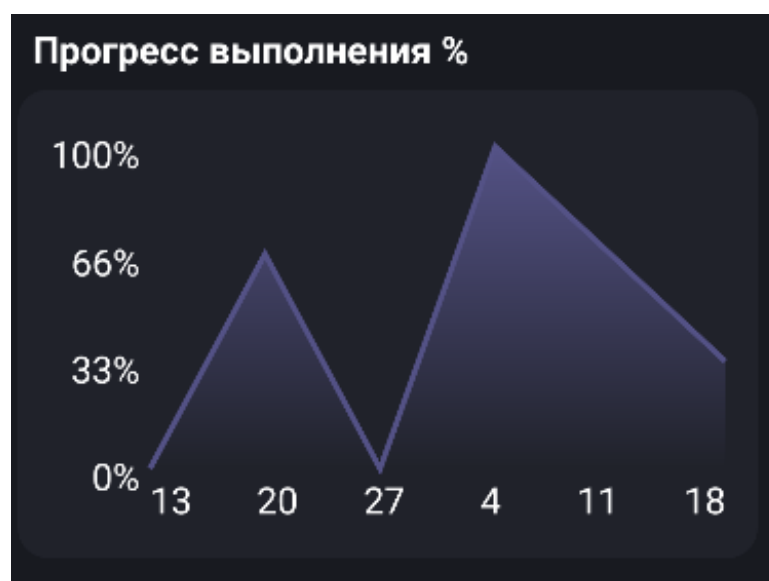


Рис. 2.21 — Процент выполнения задач и привычек (LineChart)

В будущих версиях приложения предусмотрено расширение аналитических возможностей. Среди потенциальных направлений развития рассматривается внедрение дополнительных графиков, таких как визуализация продуктивности по неделям, отдельная аналитика по задачам и привычкам, отслеживание динамики роста XP и уровней, а также график потерь жизней. Обработка и обновление данных построены следующим образом: значения для графиков формируются на основе текущего состояния базы данных Room, при изменении отметки выполнения агрегированные значения автоматически пересчитываются, а данные моментально обновляются в интерфейсе благодаря связке ViewModel и реактивным потокам LiveData или StateFlow.

Пользовательский интерфейс графиков оформлен в соответствии с принципом «понятно за 3 секунды» — структура проста и легко воспринимается даже при беглом взгляде.

Текущие реализованные характеристики визуального оформления представлены в табл. 2.10.

Таблица 2.10

Характеристики визуального оформления статистики

Элемент	Описание
Шрифты	Крупные, контрастные, легко читаемые
Анимация	Плавная подгрузка значений при открытии
Общий стиль	Чистый, минималистичный, без перегрузки

Таким образом, система статистики в Nabochi позволяет не только отслеживать повседневную продуктивность, но и усиливает мотивацию через наглядный и динамичный визуальный отклик. Благодаря этому функционал приложения выходит за рамки обычного трекера и становится полноценным инструментом анализа привычек и поведения.

2.11 Пользовательский интерфейс

Интерфейс приложения Nabochi разрабатывался с акцентом на минимализм и интуитивную навигацию, что обеспечивает пользователю визуальное удовольствие от взаимодействия. Игровые элементы органично интегрированы в повседневный пользовательский опыт.

Главный принцип: «Максимальная полезность с минимальными усилиями пользователя».

Приложение состоит из пяти ключевых экранов, назначение которых представлено в табл. 2.11.

Таблица 2.11

Назначение основных фрагментов приложения

Фрагмент	Назначение
HomeFragment	Обзор продуктивности за день: прогресс привычек, задачи, streak, статистика выполнения.
HabitsFragment	Текущий список привычек: необходимо сделать / выполнено.

CalendarFragment	Календарный просмотр задач и привычек с возможностью отметки.
GamesFragment	Персонаж, его уровень, жизни, магазин кастомизации.
SettingsFragment	Профиль пользователя, настройки уведомлений, информация о приложении.

Ключевые экраны располагаются на NavigationBarView (нижнем меню навигации), его внешний вид представлен на рис. 2.22.

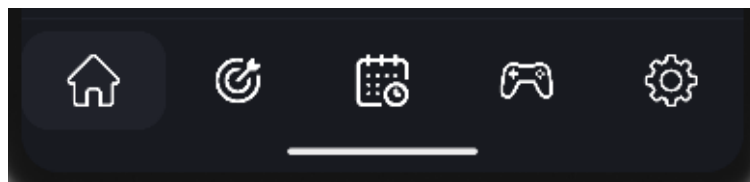


Рис. 2.22 — Нижнее меню навигации

Для наглядного представления текущего прогресса на главном экране приложения приведён блок профиля – рис. 2.23.

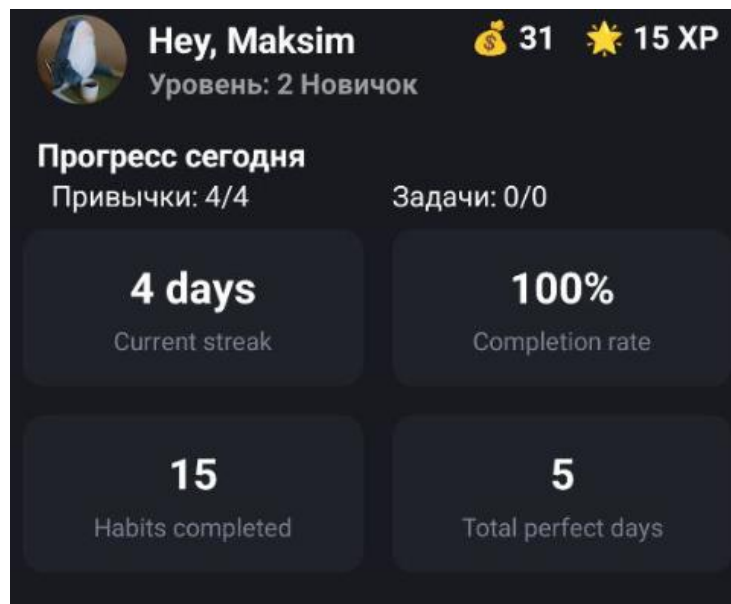


Рис. 2.23 — Блок профиля

Для удобного планирования и отметки на выбранную дату показаны следующие окна календаря и списка задач на рис. 2.24-2.25

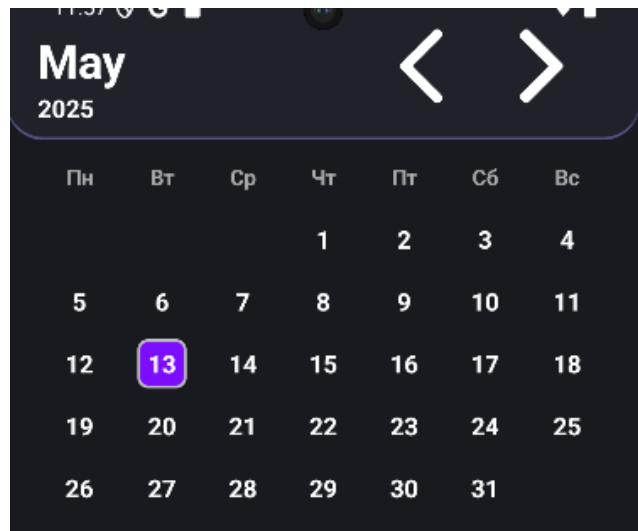


Рис. 2.24 — Блок календаря

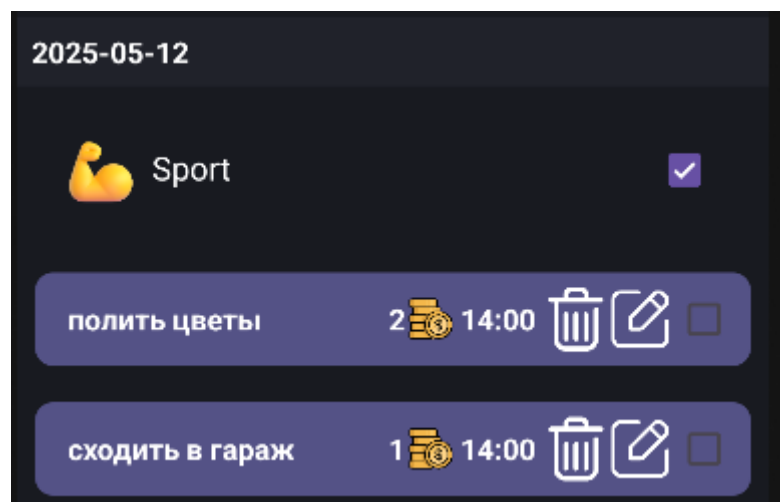


Рис. 2.25 — Задачи выбранного дня

Основной интерфейс игрового режима, где пользователь видит персонажа, уровень, опыт и жизни, представлен на рис. 2.26.



Рис. 2.26 – Экран игры

В приложении игровые элементы — жизни и опыт — реализованы в виде горизонтальных прогресс-баров, а не через привычные иконки-сердечки. Прогресс по ХР заполняется плавной анимацией при выполнении действий, что делает взаимодействие с приложением более приятным и наглядным.

Визуально прогресс-бары представлены на рис. 2.27.



Рис. 2.27 — Прогресс бары для жизней и опыта

Экран магазина позволяет изменять внешний вид персонажа, приобретая новые образы за монеты, он представлен на рис. 2.28.



Рис. 2.28 — Экран магазина

В разделе «Настройки» пользователь может включать уведомления, предоставлять системные разрешения и выходить из учётной записи - весь функционал показан на рис. 2.29–2.31.

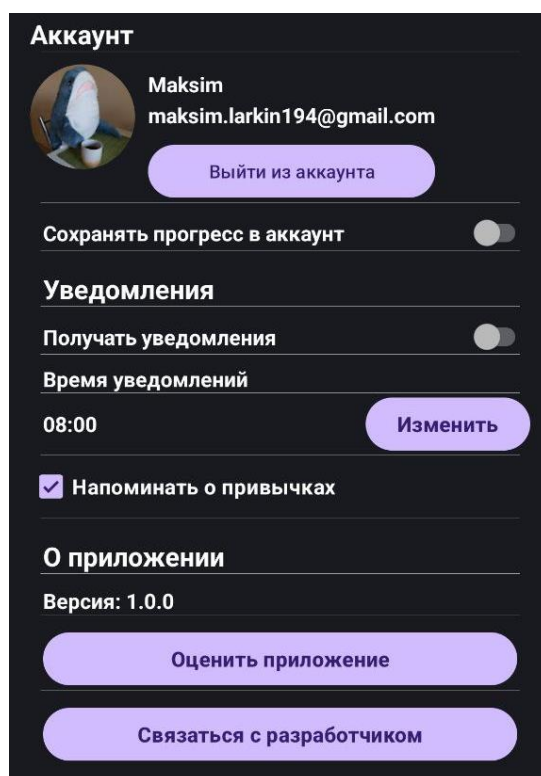


Рис. 2.29 – Общий экран настроек

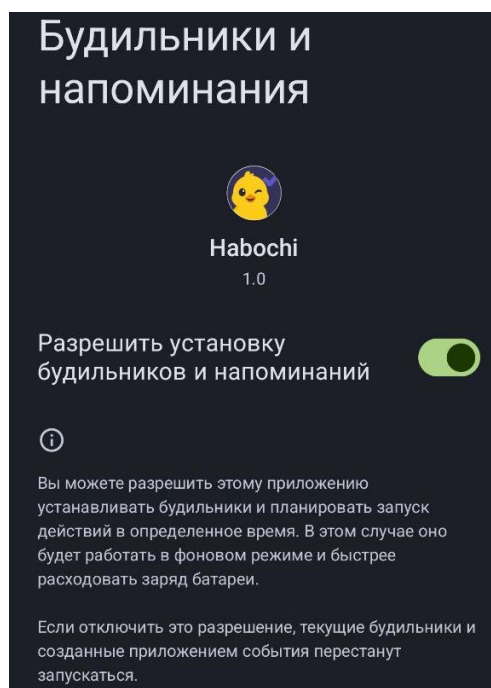


Рис. 2.30 – Разрешение на уведомления

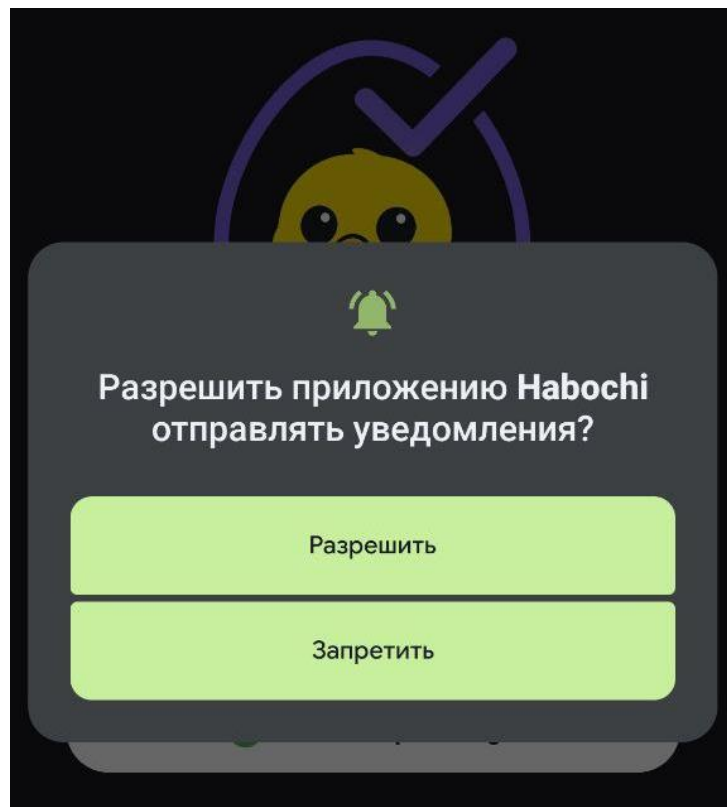


Рис. 2.31 – Запрос системных разрешений

На главном экране присутствует блок с ачивками в виде сетки карточек:

- открытые — в цвете;
- скрытые — с иконкой замка.

На рис. 2.32 показана реализация блока с ачивками.

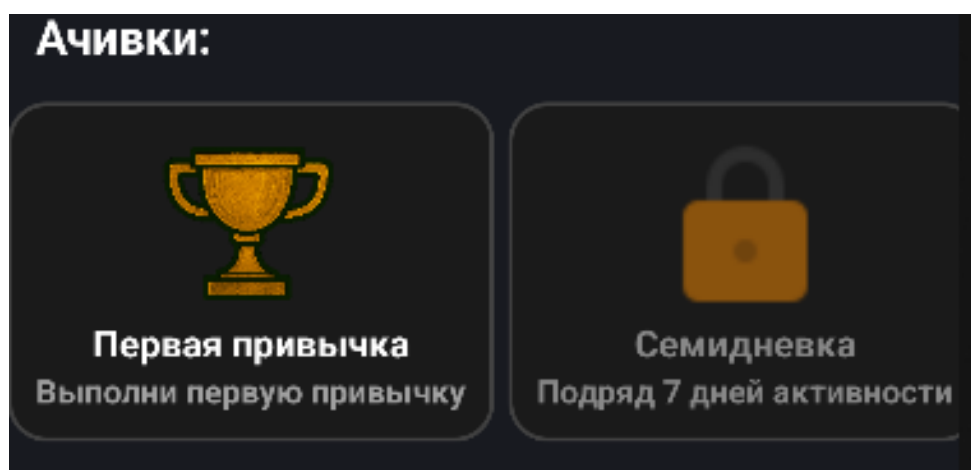


Рис. 2.32 — Блок с ачивками

Дополнительные элементы UX представлены в табл. 2.12.

Ключевые элементы пользовательского взаимодействия

Элемент	Реализация
Snackbar	Появляется при успешных действиях: например, «Добавлено +2 XP»
BottomNavigationView	Меню на 5 иконок с доступом к основным экранам
Аватар	Загружается из аккаунта Google
VectorDrawable	Используется для всех иконок
Анимации	Применяются при отметке задач, загрузке XP, открытии магазина

Пользовательский интерфейс приложения Nabochi строится на сочетании удобства и эмоциональной вовлечённости. Простота, мгновенная визуальная обратная связь и плавные анимации усиливают удовольствие от использования и способствуют формированию регулярных привычек.

2.12 Выводы по главе

В ходе проектирования и реализации мобильного приложения Nabochi была проведена комплексная работа по всем основным модулям, заявленным в постановке задачи. На основе полученных результатов можно выделить несколько ключевых итогов: реализован функциональный трекер задач и привычек с поддержкой повторяемости, ограничений по дням и визуальной отметкой; внедрены механики геймификации, такие как XP, уровни, звания, жизни, достижения и внутриигровая валюта; тщательно проработаны игровые сценарии и логика прогрессии, усиливающие мотивационный эффект; обеспечена визуальная обратная связь и анимации, поддерживающие вовлечённость пользователя; разработан блок статистики с визуализацией прогресса в формате графиков; применена архитектура MVVM, которая гарантирует чистую модульную структуру и удобство масштабирования; реализована персонализация через авторизацию с Google и обеспечена сохранность пользовательских данных; интерфейс спроектирован на основе гайдлайнов Material Design и адаптирован для повседневного использования.

Особое внимание было уделено мотивационной составляющей — приложение не просто фиксирует действия, но превращает продуктивность в игру с визуальным прогрессом и эмоциональной обратной связью. Это делает Nabochi не просто трекером, а полноценным цифровым помощником в формировании привычек и управлении задачами.

3 Управленческий анализ проекта “Nabochi”

3.1 Жизненный цикл проекта и этапы разработки

Проект Nabochi прошёл все ключевые этапы жизненного цикла ИТ-продукта, начиная от формирования концепции, до создания работающего MVP.

Разработка велась итеративными циклами с использованием канбан-подхода для гибкого управления задачами.

Процесс разработки включал следующие основные фазы:

1. Анализ и формулирование требований.

Изучались аналогичные приложения (Habitica, Duolingo, Fabulous), определялась целевая аудитория — молодые профессионалы, студенты, люди, стремящиеся к саморазвитию. Формулировались требования к функционалу: трекер привычек, геймификация, внутренняя валюта, визуальная прогрессия.

2. Проектирование архитектуры и интерфейсов

Был выбран архитектурный паттерн MVVM, спроектированы основные модели данных (UserEntity, TaskEntity, HabitEntity, AchievementEntity). Прорабатывались вопросы UX/UI — структура фрагментов, навигация, элементы кастомизации.

3. Реализация и программирование

Разрабатывались ключевые модули: трекер задач и привычек, система XP и уровней, магазин кастомизации. Интегрировалась авторизация через Google с помощью Firebase Authentication, реализовывались анимации, прогресс-бары, достижения для визуальной обратной связи.

4. Тестирование и отладка

Функционал тестировался вручную на различных сценариях, исправлялись баги, оптимизировалась производительность. Проверялась сохранность данных при повороте экрана и перезапуске приложения.

5. Улучшение и подготовка к публикации

Внедрялась базовая аналитика (подсчёт выполненных действий), дорабатывался интерфейс магазина, оптимизировался UX, велась подготовка к публикации в Google Play (запланировано на будущее).

В процессе разработки использовался канбан-подход, обеспечивающий гибкость и наглядность. Все задачи распределялись по стадиям: To Do, In Progress, Review, Done. Для трекинга применялись Trello и Notion. Разработка велась итерациями: в каждую итерацию входили проектирование, реализация и тестирование конкретной функции.

Пример канбан-доски, использованной для визуального контроля задач в проекте приведен на рис. 3.1.

To Do	In Progress	done
Аналитика по неделям	Добавление новых скинов	График статистики
Синхронизация данных в аккаунте	Добавление новых анимаций	Авторизация через Google

Рис. 3.1 — Kanban доска

Таким образом, проект развивался в соответствии с классическими принципами управления ИТ-продуктами, адаптированными под индивидуальную разработку. Гибкий итеративный подход в сочетании с визуальными методами управления позволил организовать процесс эффективно, обеспечить выполнение всех ключевых задач и достичь готовности MVP без критических задержек. Это создаёт надёжную основу для дальнейшего масштабирования функционала и выхода на более широкую аудиторию.

3.2 SWOT-анализ проекта Nabochi

Для оценки текущего состояния проекта Nabochi был проведён SWOT-анализ, который позволил выявить его сильные и слабые стороны, а также определить внешние возможности и угрозы.

Сильные стороны (Strengths) заключаются в том, что приложение объединяет привычки, задачи и элементы геймификации, что позволяет предложить пользователю не просто планировщик, а полноценную мотивационную экосистему, где задачи, привычки и игровые элементы интегрированы в единую среду. Особое внимание уделено уникальной механике жизней и достижений: модель с жизнями и системой ачивок делает процесс формирования привычек эмоционально вовлекающим и способствует долгосрочному удержанию пользователя.

Слабые стороны (Weaknesses) связаны с ограниченностью ресурсов, так как над проектом работает один разработчик. Это ограничивает скорость внедрения новых функций и увеличивает риски задержек в развитии продукта. Кроме того, магазин кастомизации на момент завершения текущего этапа развития функционирует лишь в базовом режиме и не имеет полного ассортимента предметов, что снижает глубину пользовательского вовлечения.

Возможности (Opportunities) для роста и развития проекта включают расширение монетизации за счёт разработки новых элементов кастомизации, таких как аксессуары и фоны. Это позволит повысить вовлечённость пользователей и внедрить элементы внутриигровых покупок. Ещё одной возможностью является интеграция с системными календарями и социальными сервисами, что откроет дополнительные сценарии использования приложения.

Угрозы (Threats) связаны с высокой конкуренцией на рынке приложений для продуктивности: существует риск быть вытеснённым более крупными или известными продуктами, такими как Habitica или Todoist. Кроме того, потеря интереса пользователей возможна при отсутствии развития: если не

поддерживать достаточную динамику обновлений и не добавлять новый контент, пользовательская база может постепенно сокращаться.

SWOT-матрица проекта Nabochi отображена в табл. 3.1.

Таблица 3.1

SWOT-матрица проекта

	Внутренние факторы	Внешние факторы
Позитивные	Strengths	Opportunities
	– Комбинация привычек, задач и геймификации – Уникальная механика жизней и ачивок	– Расширение монетизации – Интеграция с календарями
Негативные	Weaknesses	Threats
	– Ограниченность ресурсов – Частичная реализация магазина	– Высокая конкуренция – Риск снижения интереса пользователей

SWOT-анализ проекта Nabochi показывает наличие прочной основы для дальнейшего роста за счёт сильной концепции и востребованной ниши. При этом необходимо активно использовать внешние возможности и минимизировать риски за счёт постоянного развития функционала и адаптации к запросам пользователей.

3.3 GROW-модель

Для систематизации стратегии развития проекта Nabochi была использована методика GROW, основанная на определении цели, анализа текущего положения, рассмотрении возможных вариантов действий и планировании конкретных шагов.

Goal (Цель). Цель проекта на текущем этапе — запуск стабильной версии MVP, включающей базовые игровые механики: трекер привычек и задач, систему XP и уровней, достижения, магазин кастомизации.

Формирование полноценного MVP необходимо для тестирования интереса пользователей и получения первых отзывов до масштабного развития продукта.

Reality (Реальность). На данный момент в приложении реализованы трекеры привычек и задач, система XP и уровней, базовая статистика прогресса, а также персонаж с визуализацией состояния.

Приложение функционирует стабильно, но часть запланированного функционала (расширенная кастомизация, социальные функции) находится в разработке.

Options (Варианты развития). Для дальнейшего развития проекта рассматривались следующие направления: расширение ассортимента кастомизации персонажа за счёт добавления новых скинов и аксессуаров, внедрение социальных функций, например, возможность обмена достижениями между пользователями, а также интеграция с системными календарями для расширения сценариев использования. Каждый из этих вариантов развития предполагает дополнительное усиление мотивационной составляющей приложения.

Will (Действия и план). На ближайший этап развития поставлены следующие задачи: доработка магазина скинов и интерфейса кастомизации, улучшение визуальной составляющей — анимаций, графики и персонажей, подготовка к публикации приложения в Google Play, а также сбор обратной связи от первых пользователей, чтобы спланировать следующий этап развития.

Обобщённое представление модели GROW приведено в табл. 3.2.

Таблица 3.2

GROW модель

Этап	Содержание
Goal	Запуск стабильного MVP с ключевыми игровыми механиками
Reality	Реализованы трекеры, XP, статистика, персонаж

Options	Добавление кастомизации, скинов, социальных функций
Will	Доработка магазина, улучшение визуала, публикация в Google Play

Применение GROW-модели позволило чётко сформулировать цели развития проекта, зафиксировать текущее состояние, наметить возможные пути роста и определить конкретные шаги для реализации стратегии. Это обеспечивает последовательное развитие приложения с учётом как внутренних возможностей, так и внешних требований рынка.

3.4 GAP-анализ

Для планирования дальнейшего развития приложения Nabochi был проведён GAP-анализ. Он позволил определить текущее состояние проекта, желаемое целевое состояние и выявить существующие промежутки (GAP), требующие закрытия.

На момент завершения работы над MVP в проекте реализованы система опыта (XP) и уровней, базовые трекеры привычек и задач, графическое отображение прогресса в виде графиков и прогресс-баров, а также внутриигровая анимация и визуальная обратная связь.

Для формирования полнофункционального продукта и увеличения вовлечённости пользователей требуется внедрить систему питомцев и аксессуаров, расширить возможности кастомизации персонажа за счёт новых скинов, фонов и декоративных элементов, реализовать резервное копирование данных (облачный бэкап), а также добавить социальные функции, такие как рейтинги и совместные streak-челленджи.

В результате анализа были выявлены следующие ключевые пробелы: пока не реализована полная кастомизация персонажа, отсутствует продвинутая аналитика пользовательского прогресса, а также не внедрена

модель монетизации, которая предполагала бы покупки через внутриигровую валюту или реальные средства.

Эти промежутки определяют вектор развития продукта на ближайшие итерации.

Примерные результаты GAP-анализа представлены в табл. 3.3.

Таблица 3.3

GAP-анализ

Текущее состояние	Желаемое состояние	Промежуток (GAP)
Реализованы ХР, трекеры задач и привычек, графики, анимации	Питомцы, расширенная кастомизация, бэкап данных, социальные функции	Неполная кастомизация, отсутствие аналитики, отсутствие монетизации

Проведённый GAP-анализ позволяет наметить чёткий план развития проекта: приоритетным направлением является углубление кастомизации, усиление эмоциональной привязанности пользователя и создание экономической модели монетизации без нарушения игрового баланса.

3.5 Матрица МакКинзи

Для определения порядка реализации функциональных компонентов приложения Nabochi была использована модель приоритизации, основанная на принципах матрицы МакКинзи. В её основе лежит анализ функций по двум осям: полезность для пользователя и затраты на реализацию (временные, технические и ресурсные).

На основе данной модели все функции были распределены по четырём секторам приоритетности. Это позволило обоснованно определить, какие элементы стоит реализовать в первую очередь, а какие — отложить на более поздние этапы развития.

Наиболее эффективными по соотношению ценности и затрат стали:

- достижения (система ачивок);
- графики статистики выполнения задач и привычек.

Они обеспечивают дополнительную мотивацию пользователей при относительно низких усилиях на реализацию.

Функции с высокой полезностью, но высокими затратами (например, магазин кастомизации, питомцы) запланированы на последующие версии. Их внедрение потребует существенной переработки логики и интерфейса, но они обладают большим потенциалом вовлечения и монетизации.

Онлайн-синхронизация, как функция с высокой трудоёмкостью и относительно низким приоритетом на этапе MVP, отнесена к отложенным направлениям.

На рис. 3.2 приведены приоритеты разработки функций по матрице МакКинзи.

	Низкие затраты	Высокие затраты
Высокая полезность	Достижения, графики	Магазин кастомизации, питомцы
Низкая полезность	-	Онлайн-синхронизация

Рис 3.2 – Матрица МакКинзи

Применение модели МакКинзи позволило объективно определить наиболее рациональный порядок реализации функций. Такой подход обеспечивает эффективное распределение ограниченных ресурсов проекта и фокус на элементах, наиболее значимых для пользовательского опыта.

3.6 Метрики успеха проекта

Для анализа достигнутых результатов и последующего стратегического планирования развития приложения Nabochi были определены ключевые показатели эффективности (KPI). Они позволяют комплексно оценить не

только техническую стабильность и функциональность продукта, но и уровень вовлечённости пользователей, успешность внедрённых геймификационных элементов и перспективность монетизации.

В табл. 3.4 приведены ключевые показатели эффективности проекта.

Таблица 3.4

КРІ проекта

Метрика	Описание	Цель измерения
Удержание пользователей (Retention Rate)	Доля пользователей, возвращающихся в приложение через 1, 7, 30 дней	Оценка привлекательности и регулярности использования
Среднее количество действий в день	Количество выполненных задач/привычек в день на одного пользователя	Оценка активности и вовлечённости
Количество полученных достижений	Среднее число ачивок, разблокированных пользователем	Оценка мотивационной эффективности геймификации
Частота покупок жизней и скинов	Количество покупок внутриигровых элементов на пользователя	Оценка интереса к системе наград и кастомизации

Анализ представленных метрик позволяет сделать следующие выводы. Высокий показатель удержания пользователей (retention) указывает на значимость приложения для целевой аудитории и эффективность его интерфейса. Среднее количество выполняемых действий в сутки отражает степень взаимодействия с основными функциями, а также может сигнализировать о полезности геймифицированных стимулов. Активность по получению достижений демонстрирует работу поведенческой модели мотивации. Наконец, частота покупок в магазине служит индикатором привлекательности системы наград и глубины вовлечённости в персонализацию.

Таким образом, представленные КРІ позволяют количественно отслеживать поведение пользователей, оценивать работу мотивационных и

игровых механизмов, а также принимать обоснованные решения о доработке функциональности и дальнейшей оптимизации приложения.

3.7 Выводы по главе

В ходе управленческого анализа проекта Nabochi были применены различные методы стратегического планирования и оценки развития: SWOT-анализ, GAP-анализ, GROW-модель, матрица приоритетов МакКинзи, а также система ключевых показателей эффективности (KPI). Применение этих инструментов позволило чётко сформулировать сильные и слабые стороны проекта, выявить существующие промежутки (GAP) между текущим состоянием приложения и целевыми ориентирами, а также определить реалистичные пути развития функциональности с учётом имеющихся ресурсов. Кроме того, удалось установить приоритеты дальнейших разработок на основе баланса полезности и затрат, а также наметить метрики для объективной оценки успеха и вовлечённости пользователей.

Анализ подтвердил, что Nabochi обладает устойчивой концепцией и высокой потенцией к развитию, благодаря уникальному сочетанию трекинга продуктивности и геймификационных механизмов. При этом выявлены направления, требующие внимания: расширение кастомизации, внедрение социальных функций и развитие монетизационной модели.

Таким образом, управление проектом осуществлялось не только на уровне реализации функционала, но и с позиций стратегического развития, что создаёт прочную основу для дальнейшего масштабирования и успешного выхода на рынок.

Заключение

В рамках бакалаврской работы была поставлена цель — разработка мобильного приложения Nabochi, сочетающего в себе функции трекера задач и привычек с элементами геймификации, направленными на повышение мотивации и вовлечённости пользователей. Для её достижения были последовательно решены следующие задачи: проведён обзор существующих решений в области цифровой продуктивности и игровых механик; сформулированы функциональные и нефункциональные требования к разрабатываемому продукту; спроектирована архитектура приложения на основе паттерна MVVM с применением современных технологий Android-разработки (Room, DataStore, Firebase Authentication); реализованы ключевые модули, включая отслеживание привычек и задач, систему опыта (XP) и уровней, магазин визуальной кастомизации, блок достижений и аналитики; внедрена система персонализации на основе входа через Google-аккаунт; выполнено тестирование корректности работы функциональных компонентов.

Разработанный программный продукт обеспечивает пользователю не только возможность планировать и фиксировать ежедневные действия, но и получить дополнительную мотивацию за счёт игровых элементов, визуального прогресса и системы наград. Такой подход способствует формированию устойчивых поведенческих паттернов и регулярному взаимодействию с приложением.

Практическая значимость разработки заключается в создании функционального и визуально привлекательного инструмента самоорганизации, ориентированного на широкий круг пользователей. Закладка архитектурных решений обеспечивает масштабируемость проекта и его готовность к будущему развитию.

В перспективе планируется расширение системы кастомизации за счёт добавления новых скинов, фонов и элементов оформления. Также

предполагается внедрение социальных функций, таких как достижения в группе, рейтинги и совместные streak-челленджи. Кроме того, планируется подключение облачной синхронизации и резервного копирования данных, а также развитие монетизационной модели и новых форм игровой активности.

Таким образом, поставленная цель достигнута, все ключевые задачи решены, а созданное приложение обладает высоким потенциалом для дальнейшего совершенствования и коммерческого внедрения.

Список использованных источников

1. Android Developers. Официальная документация по архитектуре приложений [Электронный ресурс]. — URL: <https://developer.android.com/jetpack/guide> (дата обращения: 15.01.2025).
2. Android Developers. Room Persistence Library [Электронный ресурс]. — URL: <https://developer.android.com/topic/libraries/architecture/room> (дата обращения: 15.01.2025).
3. Android Developers. DataStore: New Data Storage Solution [Электронный ресурс]. — URL: <https://developer.android.com/topic/libraries/architecture/datastore> (дата обращения: 15.01.2025).
4. Google Firebase. Официальная документация по аутентификации [Электронный ресурс]. — URL: <https://firebase.google.com/docs/auth> (дата обращения: 15.01.2025).
5. Дейтел П., Дейтел Х. Как программировать на Kotlin. — М.: Вильямс, 2020. — 624 с.
6. Геймификация: как повысить вовлеченность пользователей / Ю. Вербах, С. Хантер. — М.: Манн, Иванов и Фербер, 2021. — 256 с.
7. Nielsen J. Usability Engineering. — San Francisco: Morgan Kaufmann, 1994. — 362 p.
8. Statista. Usage of habit tracking apps worldwide [Электронный ресурс]. — URL: <https://www.statista.com/statistics/1219881/habit-tracking-app-usage-worldwide/> (дата обращения: 15.01.2025).
9. Habitica Wiki. Руководство по геймификации привычек [Электронный ресурс]. — URL: https://habitica.fandom.com/wiki/Habitica_Wiki (дата обращения: 15.01.2025).
10. Android Developers. Best Practices for UX Design [Электронный ресурс]. — URL: <https://developer.android.com/guide/topics/ui/ux-guidelines> (дата обращения: 15.01.2025).

Приложение А

Исходные коды программы

MainActivity.kt

```
class MainActivity : AppCompatActivity() {

    private val requestPermissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted: Boolean ->
        if (isGranted) {
            Toast.makeText(this, "Разрешение на уведомления получено",
                Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "Разрешение на уведомления
отклонено", Toast.LENGTH_SHORT).show()
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val navHostFragment =
supportFragmentManager.findFragmentById(R.id.nav_host_fragment) as
NavHostFragment
        val navController = navHostFragment.navController

        val prefs = getSharedPreferences("user", Context.MODE_PRIVATE)
        val isLoggedIn = prefs.getString("username", null) != null

        val navInflater = navController.navInflater
        val graph = navInflater.inflate(R.navigation.nav_graph)
        graph.setStartDestination(if (isLoggedIn) R.id.nav_home else
R.id.loginFragment)
        navController.graph = graph

        val bottomNavigationView =
findViewById<BottomNavigationView>(R.id.bottom_navigation)
        bottomNavigationView.setupWithNavController(navController)

        bottomNavigationView.setOnItemReselectedListener { item ->
            if (item.itemId == R.id.nav_home) {
                supportFragmentManager.popBackStack()
            }
        }

        requestNotificationPermission()
    }

    private fun requestNotificationPermission() {
        if (android.os.Build.VERSION.SDK_INT >=
android.os.Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(
                this,
                Manifest.permission.POST_NOTIFICATIONS
            ) != PackageManager.PERMISSION_GRANTED) {
                requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
            }
        }
    }
}
```

Продолжение приложения А

```
        ) != PackageManager.PERMISSION_GRANTED
    ) {

requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATION
S)

    }

}

}
```

HomeFragment

```
class HomeFragment : Fragment() {
    private lateinit var userRepository: UserRepository
    private lateinit var db: AppDatabase
    private lateinit var viewModel: MainViewModel

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_home, container,
false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        db = AppDatabase.getDatabase(requireContext())
        val userDao = db.userDao()
        val habitsDao = db.habitsDao()
        val taskDao = db.taskDao()
        userRepository = UserRepository(userDao)

        viewModel = ViewModelProvider(requireActivity(),
MainViewModelFactory(userRepository, taskDao, habitsDao,
requireContext()))
            .get(MainViewModel::class.java)

        calculateStats()
        setupHabitsCompletedChart()
        setupCompletionRateChart()
        loadUserData()
        loadTodayProgress()

        lifecycleScope.launch {
            viewModel.updateLives()
            initAchievements()
            loadAchievements()
        }
    }

    private suspend fun initAchievements() {
        val dao = db.achievementDao()
    }
}
```

Продолжение приложения А

```
val existing = dao.getAll().map { it.id }.toSet()
Log.d("HomeFragment", "Existing achievements: $existing")

val predefined = listOf(
    AchievementEntity("first_task", "Первая задача", "Выполни первую задачу"),
    AchievementEntity("first_habit", "Первая привычка", "Выполни первую привычку"),
    AchievementEntity("three_in_day", "3 за день", "Выполни 3 дела за один день"),
    AchievementEntity("perfect_day", "Идеальный день", "Выполни все за день"),
    AchievementEntity("seven_streak", "Семидневка", "Подряд 7 дней активности"),
    AchievementEntity("task_master", "Мастер дел", "Выполни 50 задач"),
    AchievementEntity("habit_keeper", "Продуктивность - это я", "Выполни 30 привычек"),
    AchievementEntity("coin_collector", "Люблю золото", "Заработай 100 монет")
)

predefined.forEach { achievement ->
    if (achievement.id !in existing) {
        dao.insert(achievement)
        Log.d("HomeFragment", "Inserted new achievement: ${achievement.id}")
    }
}

private suspend fun checkAchievements(completedTasks: Int, completedHabits: Int) {
    val dao = db.achievementDao()

    if (completedTasks >= 1) unlockAchievement(dao, "first_task")
    if (completedHabits >= 1) unlockAchievement(dao, "first_habit")
    if ((completedTasks + completedHabits) >= 3) unlockAchievement(dao, "three_in_day")

    val today = getToday()
    val allTasksToday = db.taskDao().getTasksByDate(today)
    val allHabits = db.habitsDao().getAllHabits()
    val completedHabitIdsToday = db.habitCompletionDao().getCompletedHabitsOnDate(today).map { it.habitId }

    val activeHabitsToday = allHabits.filter { habit ->
        val startDate = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault()).parse(habit.startDate)
        if (startDate == null || startDate.time > System.currentTimeMillis()) return@filter false
        when (habit.repeatType) {
            RepeatType.DAILY -> true
        }
    }
```

Продолжение приложения А

```
RepeatType.WEEKLY -> {
    val dayOfWeek =
Calendar.getInstance().get(Calendar.DAY_OF_WEEK) - 1
    habit.repeatDays?.contains(dayOfWeek) == true
}
}

val allDone = allTasksToday.all { it.isCompleted } &&
    activeHabitsToday.all { it.id in
completedHabitIdsToday }
    if (allDone && allTasksToday.isNotEmpty())
unlockAchievement(dao, "perfect_day")

val user = userRepository.getUser()
// Семидневка
val completedHabitsDates =
db.habitCompletionDao().getAllCompletedDates().map { it.date }
    val completedTasksDates = db.taskDao().getAllTasks().filter {
it.isCompleted }.map { it.date }

val currentStreak =
calculateCurrentStreak(completedHabitsDates, completedTasksDates)
    if (currentStreak >= 7) unlockAchievement(dao, "seven_streak")

// Мастер дел
val allTasks = db.taskDao().getAllTasks()
    if (allTasks.count { it.isCompleted } >= 50)
unlockAchievement(dao, "task_master")

// Привычка – вторая натура
val allHabitCompletions =
db.habitCompletionDao().getAllCompletedDates()
    if (allHabitCompletions.size >= 30) unlockAchievement(dao,
"habit_keeper")
    // Люблю золото
    if (user.coins >= 100) unlockAchievement(dao,
"coin_collector")
}

private suspend fun unlockAchievement(dao: AchievementDao, id:
String) {
    val achievement = dao.getAll().find { it.id == id &&
!it.isUnlocked } ?: return
    dao.update(achievement.copy(isUnlocked = true, unlockDate =
getToday()))
}
private fun loadAchievements() {
    val recycler =
view?.findViewById<RecyclerView>(R.id.rv_achievements)
    recycler?.layoutManager =
LinearLayoutManager(requireContext(), LinearLayoutManager.HORIZONTAL,
false)
```

Продолжение приложения А

```
lifecycleScope.launch {
    val achievements = db.achievementDao().getAll()
        .sortedWith(compareByDescending<AchievementEntity> {
it.isUnlocked }
            .thenByDescending { it.unlockDate ?: "" })
    Log.d("HomeFragment", "Loaded achievements:
    ${achievements.map { "${it.id} (unlocked=${it.isUnlocked})" }}")
    requireActivity().runOnUiThread {
        recycler?.adapter = AchievementAdapter(achievements)
    }
}

private fun getToday(): String {
    return SimpleDateFormat("yyyy-MM-dd",
Locale.getDefault()).format(Date())
}

fun loadUserData() {
    lifecycleScope.launch {
        val user = userRepository.getUser()
        Log.d("HomeFragment", "Обновляем UI: монеты=${user.coins},
XP=${user.xp}")
        val firebaseUser = FirebaseAuth.getInstance().currentUser
        requireActivity().runOnUiThread {
            val name = firebaseUser?.displayName ?: "Гость"
            view?.findViewById<TextView>(R.id.userName)?.text =
"Hey, $name"

            firebaseUser?.photoUrl?.let { url ->
                val imageView =
view?.findViewById<ImageView>(R.id.profileImage)
                Glide.with(this@HomeFragment)
                    .load(url)
                    .circleCrop()
                    .into(imageView!!)
            }
        }
    }
}
```


Приложение Б

Презентационный материал



БАКАЛАВРСКАЯ РАБОТА

на тему «Разработка мобильного приложения для улучшения продуктивности и качества жизни»

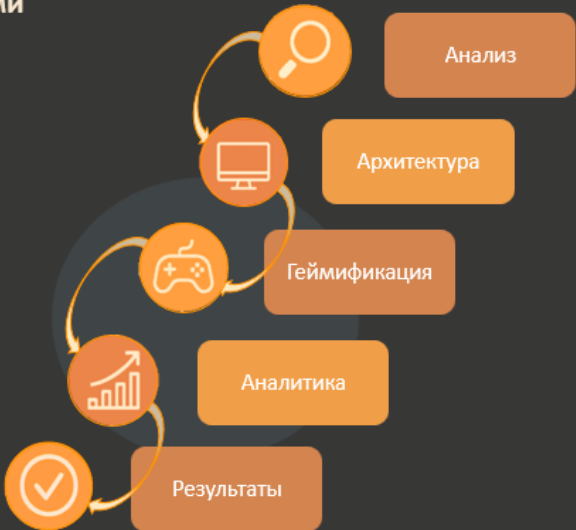
Кафедра программной инженерии
Студент группы РПИС-11: Ларкин М.Л.
Руководитель ВКР: ст. препод. каф. При Расеева Е.В

Самара – 2025 г.

Цель: создание Android-приложения «Nabochi» — трекер привычек и задач с элементами геймификации.

Задачи:

- Анализ рынка и пользовательских потребностей
- Разработка архитектуры
- Реализация геймифицированных механик
- Внедрение системы аналитики и кастомизации
- Тестирование и анализ результатов



```
graph TD; A[Анализ] --> B[Архитектура]; B --> C[Геймификация]; C --> D[Аналитика]; D --> E[Результаты]; E --> F[Проверка];
```

Продолжение приложения Б

Актуальность и новизна

Проблема аналогов:

- Потеря мотивации
- Нехватка “эмоций” и прогресса

Решение:

- Геймификация
- Кастомизация
- Визуальный отклик

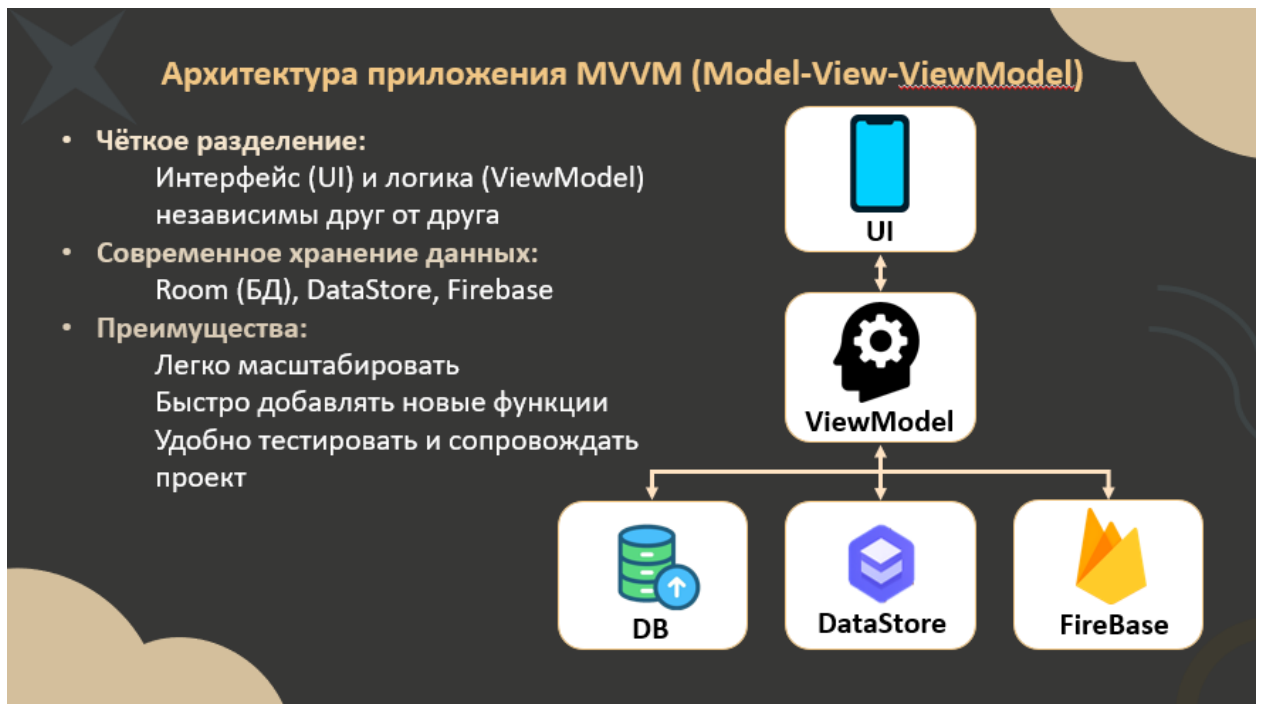
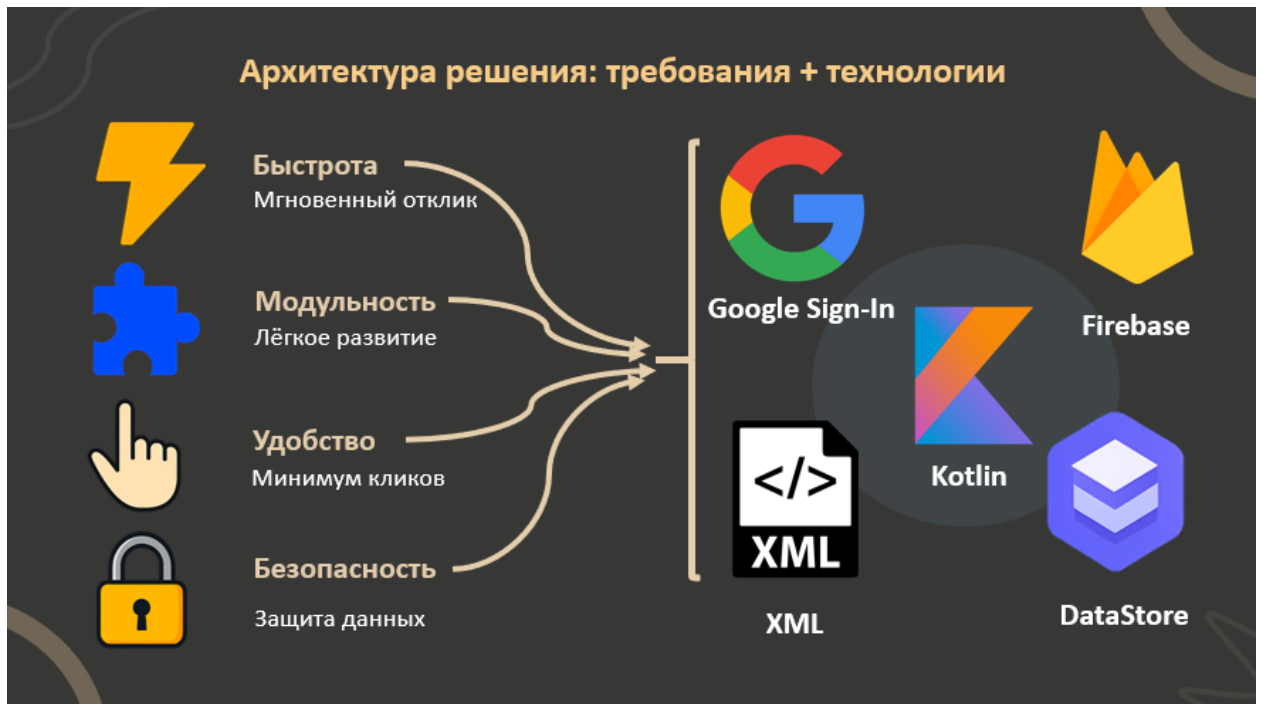


Анализ аналогов

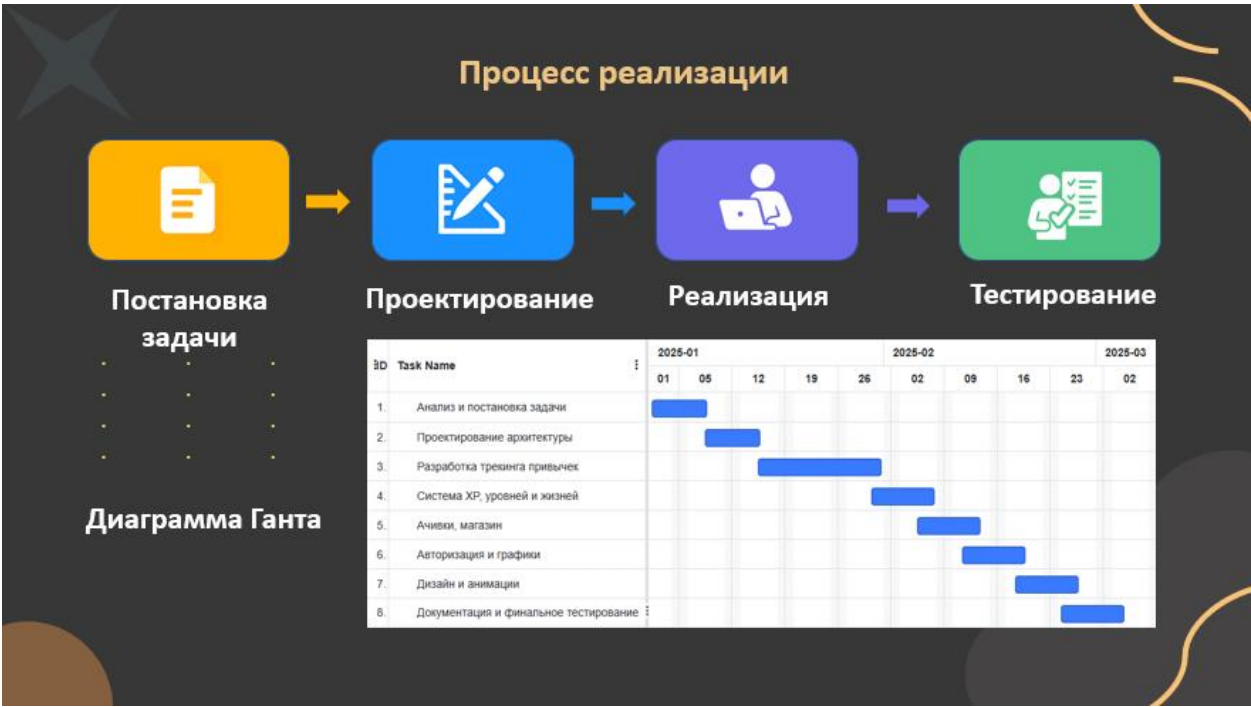
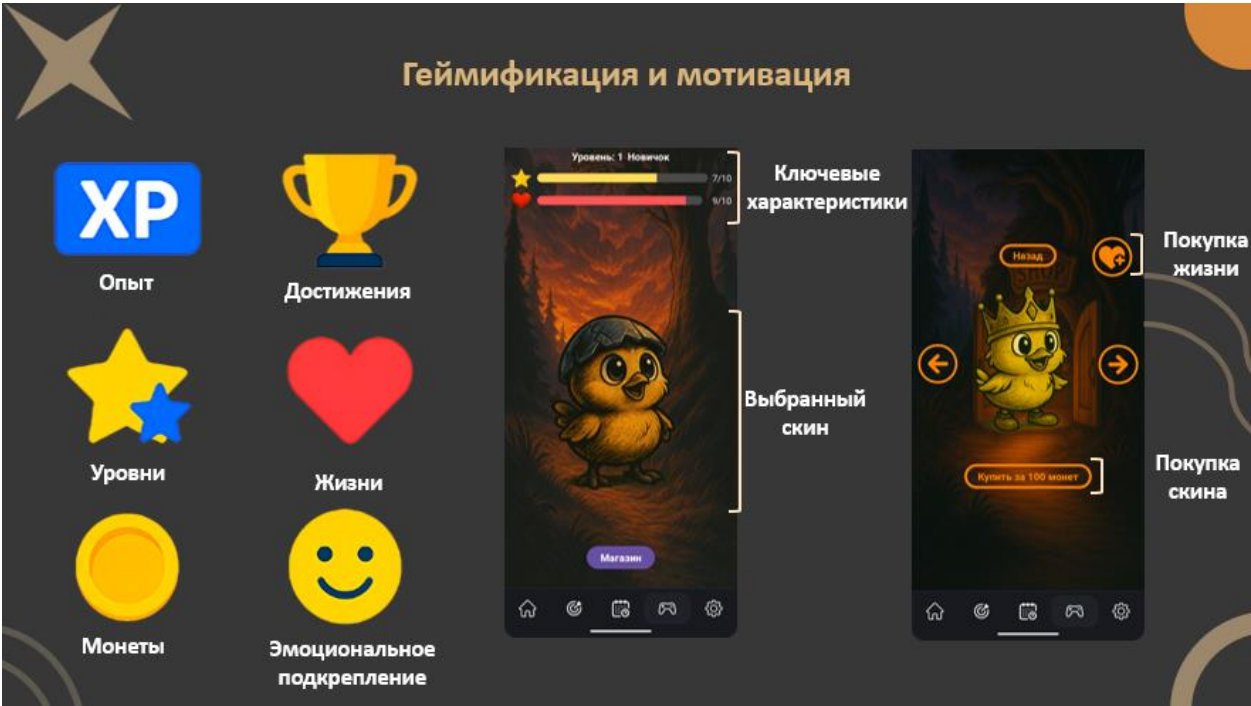
	 Habitica	 Todoist	 Fabulous
Задачи	✓	✓	✓
Привычки	✓	✗	✓
Геймификация	✓	✗	✗
Удобные графики	✗	✓	✗
Простой UX	✗	✓	✗

Нет ни одного приложения, сочетающего легкий интерфейс, привычки, кастомизацию и геймификацию.

Продолжение приложения Б



Продолжение приложения Б



Продолжение приложения Б



Выводы

Приложение реализовано,
Цель достигнута,
Задачи решены

Nabochi — рабочий инструмент для
продуктивности

Спасибо за внимание!

